



Bases de la modélisation objet *avec UML (Unified Modeling Language)*

Pr. Jean-Marc Jézéquel

IRISA - Univ. Rennes I

Campus de Beaulieu

F-35042 Rennes Cedex

e-mail : jezequel@irisa.fr

<http://www.irisa.fr/prive/jezequel>

Twitter @jmjezequel

1

1 - Introduction

L'ingénierie du logiciel aujourd'hui

2

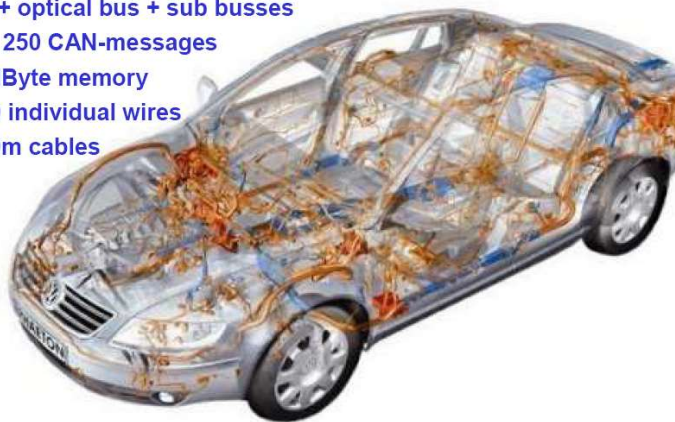
Airbus A 380



Automotive industry: Volkswagen

Phaeton

- ◆ 61 networked ECUs
- ◆ 3 bus systems + optical bus + sub busses
- ◆ 2500 signals in 250 CAN-messages
- ◆ more than 50 MByte memory
- ◆ more than 2000 individual wires
- ◆ more than 3800m cables



Google

- 500 000 serveurs
 - répartis dans une trentaine de datacenters.
- Une phénoménale capacité de calcul qui permet à Google d'assurer le fonctionnement d'un grand nombre de services
 - Google Earth ...
- répondre à plus d'1 milliard de requêtes par jour,
 - chacune interrogeant 10 milliards de pages Web
 - en moins d'un cinquième de seconde

The Google codebase includes approximately one billion files and has a history of approximately 35 million commits spanning Google's entire 18-year existence. The repository contains 86TB² of data, including approximately two billion lines of code in nine million unique source files.

5

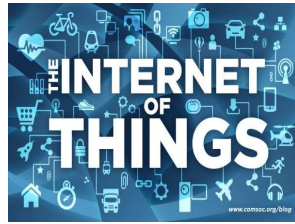
Samsung

- 1,5 milliard de smartphones en circulation
 - 300 millions de plus par an
- Des milliers de versions de logiciels
- Time-to-market ~ 3 mois



6

From Zero Power Computing ...

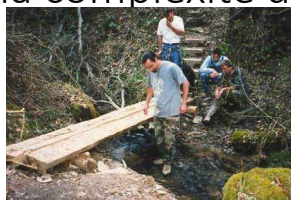


...to Systems of Systems

- Global control of a massively distributed system totaling about 10^9 LoC



Programming-in-the-Large : Gérer la complexité due à la taille

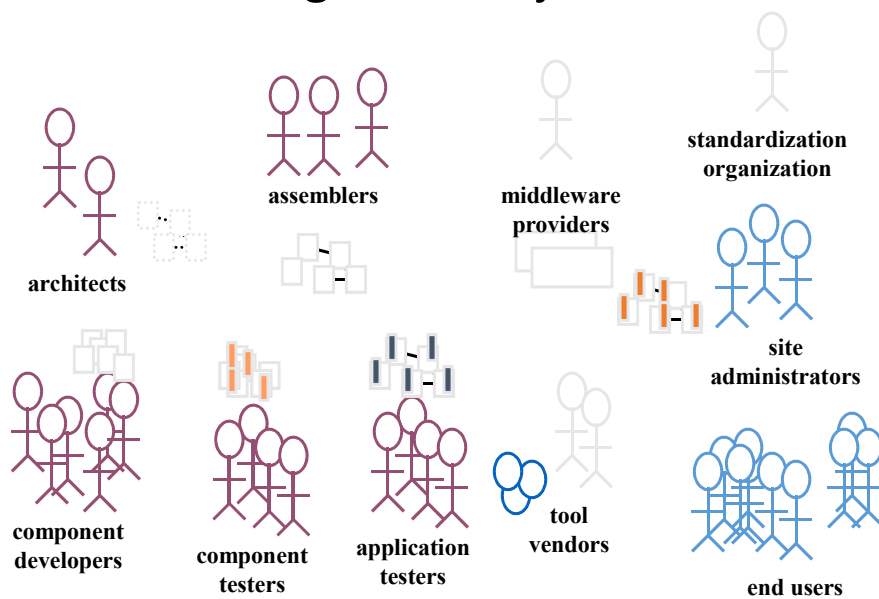


Des logiciels complexes...

- Logiciels de grande taille
 - des millions de lignes de code
 - des équipes nombreuses
 - durée de vie importante
 - des lignes de produits
 - plateformes technologiques complexes
 - évolution continue
- Logiciels complexes
- Logiciels critiques
- ...

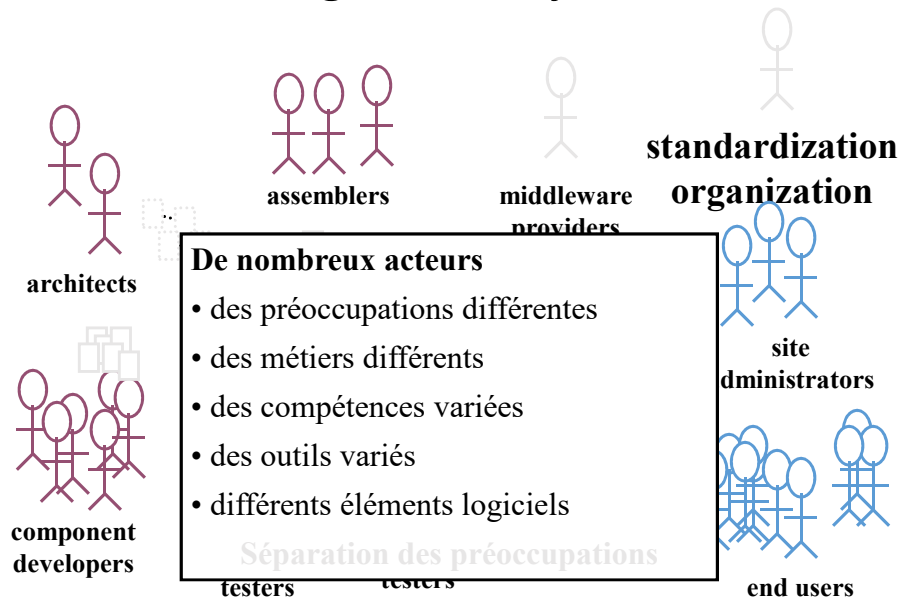
10

L'industrie logicielle aujourd'hui



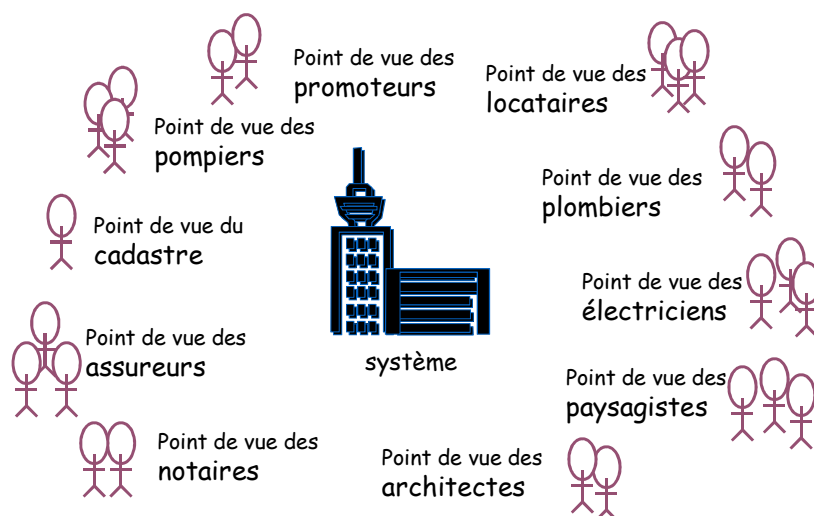
11

L'industrie logicielle aujourd'hui



12

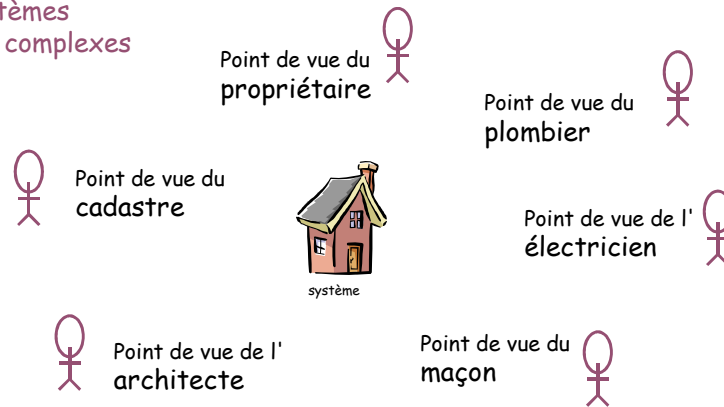
Séparations des préoccupations



13

Séparations des préoccupations

Utile même pour
des systèmes
"moins" complexes



14

Problématique

- Complexité croissante des logiciels
- Séparations des préoccupations
- Séparations des métiers
- Multiplicité des besoins
- Multiplicité des plateformes
- Évolution permanente

Logiciel = Code ?

15

16



Synthèse

- Des Modèles plutôt que du Code
 - Un modèle est la simplification/abstraction d'un aspect de la réalité
 - Nous construisons donc des modèles afin de mieux comprendre les systèmes que nous développons
 - Nous modélisons des systèmes complexes parce que nous sommes incapables de les comprendre dans leur totalité
 - Le code ne permet pas de simplifier/abstraire la réalité

18

Objectifs du cours

- Acquérir la **bases de la modélisation objet** : comment il est possible de définir un certain nombre de modèles orientés objets
 - plus ou moins abstraits
 - plus ou moins élaborés
 - mais décrivant tous une facette complémentaire du système à réaliser selon un point de vue ou un métier différent...
 - ... A fins de préparer une conception et réalisation dans un langage à objets
- L'accent est mis sur les règles de cohérence intra et inter-modèles, ainsi que sur la différence qui peut être faite entre
 - modèles conceptuels, de spécification et d'implémentation
 - modèles métiers et modèles techniques
 - modèles produits et modèles de processus.
- Le langage UML est utilisé comme support
 - Mais *ceci n'est pas un cours d'UML!*

19

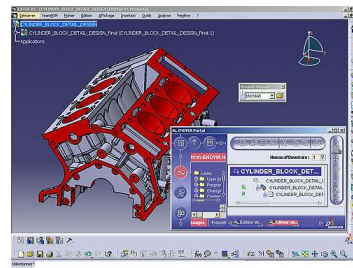
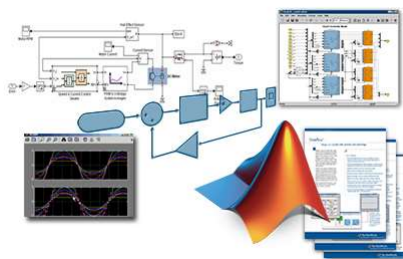
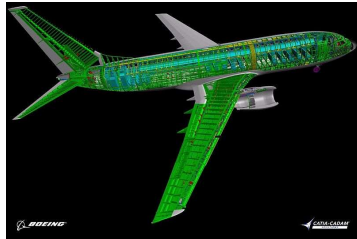
Bases de la modélisation Objet

- Ce cours introduit les notions de base de la modélisation de logiciel...
 - objets, classes, modularité, contrats, associations, généralisation/spécialisation, cas d'utilisations, diagrammes de séquence, automates...),
 - Ces notions sont illustrées dans le langage UML
- ... et déroule sur une étude de cas comment les mettre en œuvre
- Objectif : être capables, à partir d'un cahier des charges, de construire de manière systématique un modèle d'analyse bien formé exprimé en UML.
- Modalités du CC
 - QCM portant sur le cours
 - TP/mini-projet noté

2 - Pourquoi UML ?

Un langage de modélisation pour le logiciel

Langages de modélisation multiples



Welcome to Balsamiq Mockups

This is a sample mockup to get you started. Feel free to delete or reuse some of these controls for your own design.

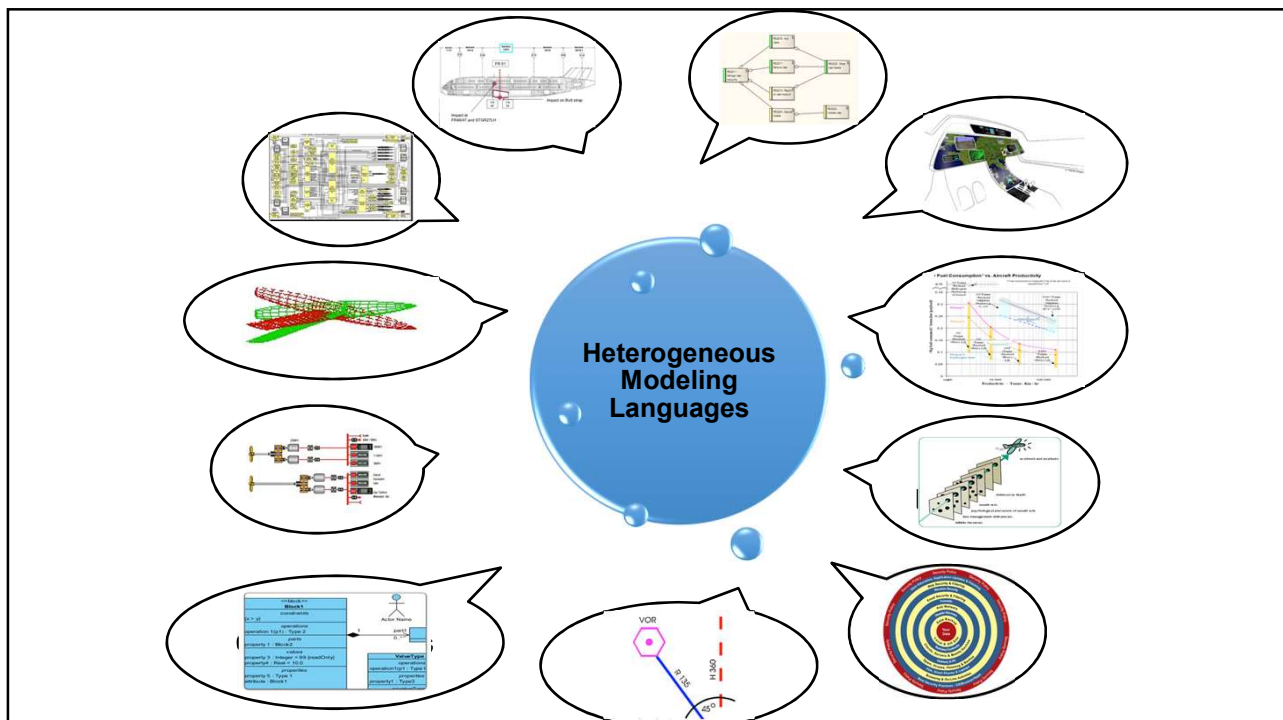
Anything you do is immediately saved on your computer, so it will be there when you come back.

There's full undo/redo, so don't be afraid to explore!

This demo version gives you a taste of Balsamiq. The demo version lets you import images from your desktop, add multiple mockups, link elements, create symbols (page markers, navigation, and reusable components) and much more.

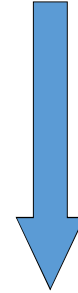


22



UML : Langage de modélisation pour le logiciel

- Réfléchir
- Définir la structure « gros grain »
- Documenter
- Guider le développement
- Développer, Tester, Auditer



24

Modélisation UML

- Modélisation selon 4 points de vue principaux :
 - Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
 - Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
 - Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
 - Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

32

Modélisation UML

- Modélisation selon 4 points de vue principaux :



- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

33

“Objet” (Définition)

- Formellement : la fermeture transitive d'une fonction
- Concrètement : encapsulation d'un état avec un ensemble d'opérations travaillant sur cet état
 - abstraction d'une entité du monde réel
 - existence temporelle :
 - création, évolution, destruction
 - identité propre à chaque objet
 - peut être vu comme une machine
 - ayant une mémoire privée et une unité de traitement,
 - et rendant un ensemble de services

34

“Classe”

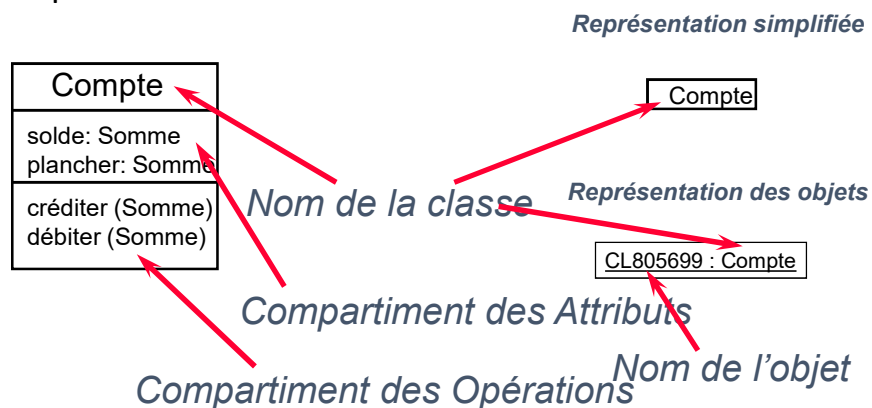
Définition en temps que type

- Implantation d'un type de donné abstrait
- Description des propriétés et des comportements communs à un ensemble d'objets
 - un objet est une instance d'une classe (eq. variable vs. type)
- Chaque classe a un nom, et un corps qui défini :
 - les attributs possédés par ses instances
 - les opérations définies sur ses instances,
 - et permettant d'accéder, de manipuler, et de modifier leurs attributs
 - une classe peut elle même être un objet (*Smalltalk*)

35

Notations UML pour classes et objets

- Représentation d'une classe



36

Classe vs. Objets

Une **classe** spécifie la structure et le comportement d'un ensemble d'objets de même nature

- La structure d'une classe est constante

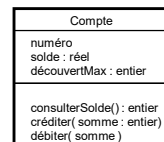


Diagramme de classes

M1

M0

- Des **objets** peuvent être ajoutés ou détruits pendant l'exécution
- La valeur des attributs des objets peut changer

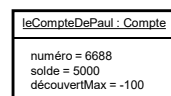
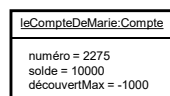
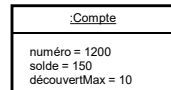


Diagramme d'objets



37

“Classe”

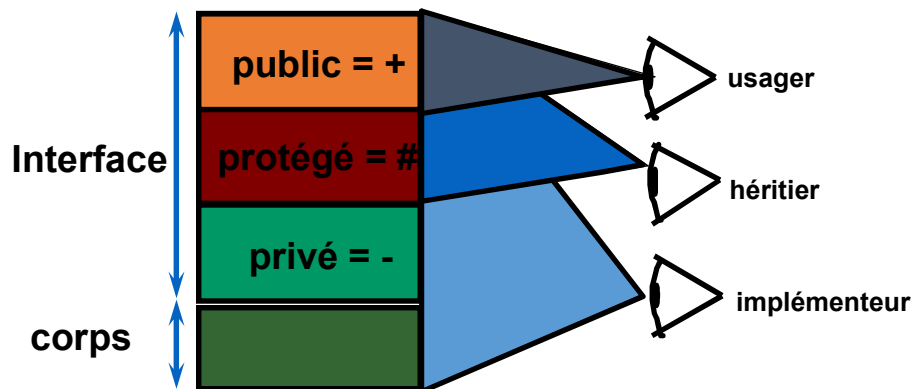
Définition en temps que module

- Modularité = Gestion complexité des systèmes
- Concept présent depuis longtemps en informatique
 - sous-routines, unité de compilation (le fichier en C)
- Notion de module intégrée aux langages
 - Modula-2 (module), Ada83 (package), puis lang. à objets
- Favorise :
 - masquage d'information (abstraction)
 - encapsulation (facilite les modifications à portée locale)
 - dissociation interface/implantation=> composant réutilisable

38

Visibilité

- Différentes visibilités des membres d'une classe



39

Modélisation UML

- Modélisation selon 4 points de vue principaux :
 - Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, [relations](#), généricité, héritage
 - Structuration en paquetages
 - Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
 - Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
 - Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

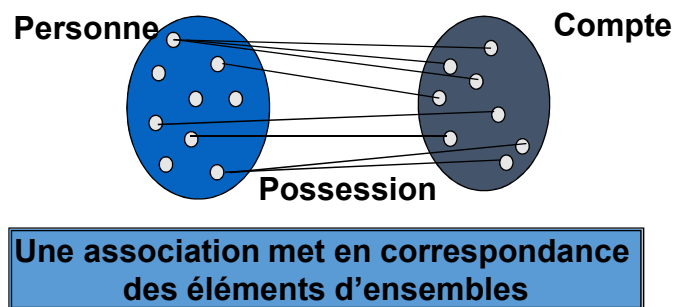
40

Relation entre classes

- Deux points de vue :
 - Une relation met en correspondance des éléments d'ensembles
 - Une relation permet la description d'un concept à l'aide d'autres concepts
- Une contrainte :
 - Une relation est un lien stable entre deux objets

41

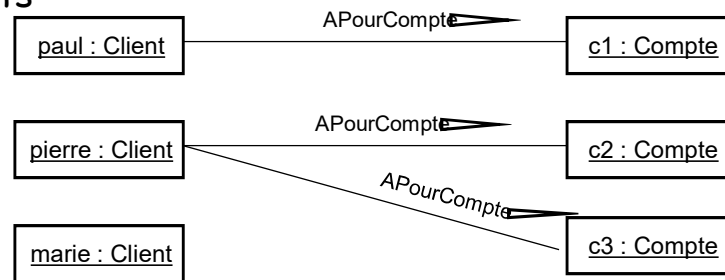
Vue ensembliste d'une relation : Graphe de la relation



42

Liens (entre objets)

Un **lien** indique une connexion entre deux objets



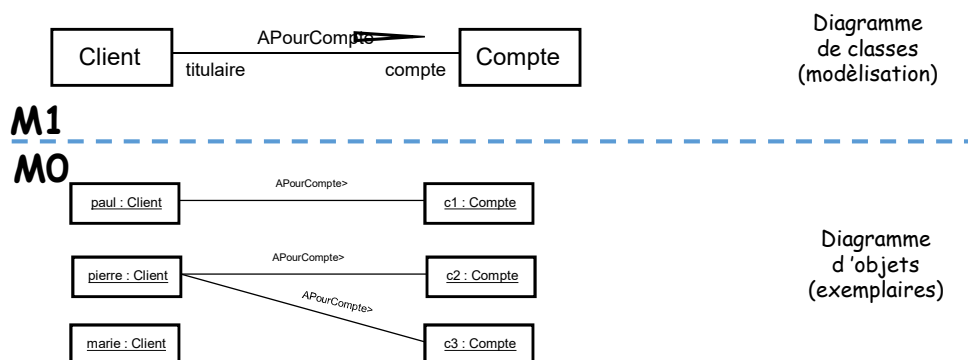
Note de style :

- les noms des liens sont des formes verbales et commencent par une majuscule
- ▬ indique le sens de la lecture (ex: « paul APourCompte c1 »)

43

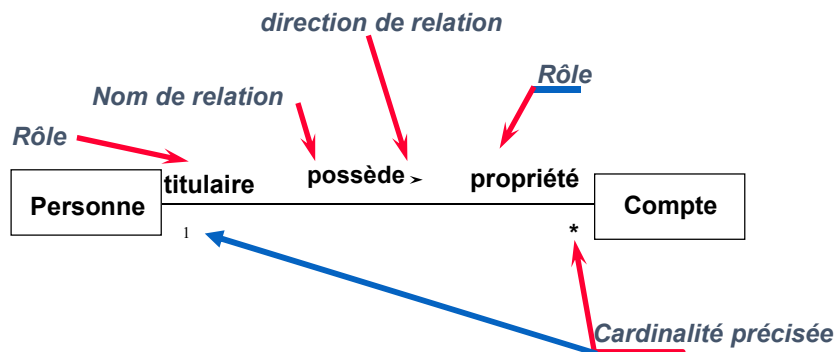
Associations (entre classes)

Une **association** décrit un ensemble de liens de même "sémantique"



44

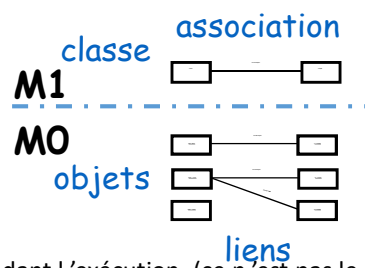
Représentation des associations : direction, rôle, cardinalité



45

Association vs. Liens

- Un **lien** lie deux **objets**
- Une **association** lie deux **classes**
- Un **lien** est une instance d'**association**
- Une **association** décrit un ensemble de **liens**

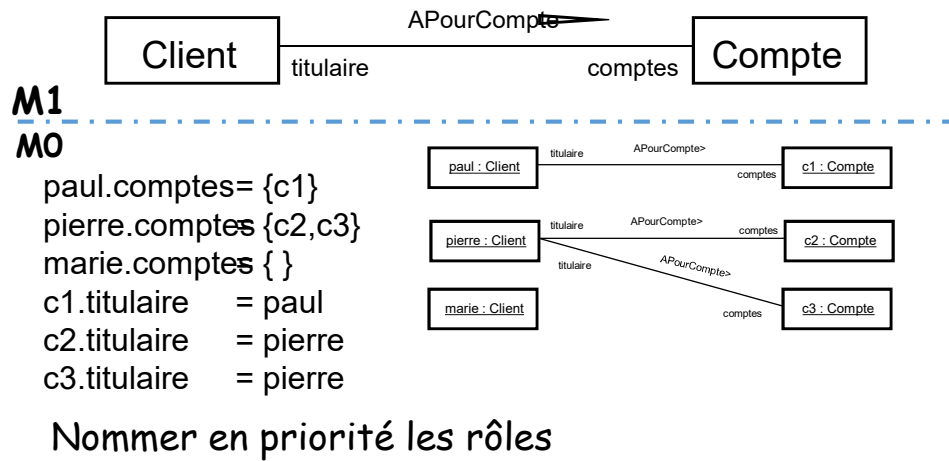


- Des **liens** peuvent être ajoutés ou détruits pendant l'exécution, (ce n'est pas le cas des associations)

Le terme "relation" ne fait pas partie du vocabulaire UML

46

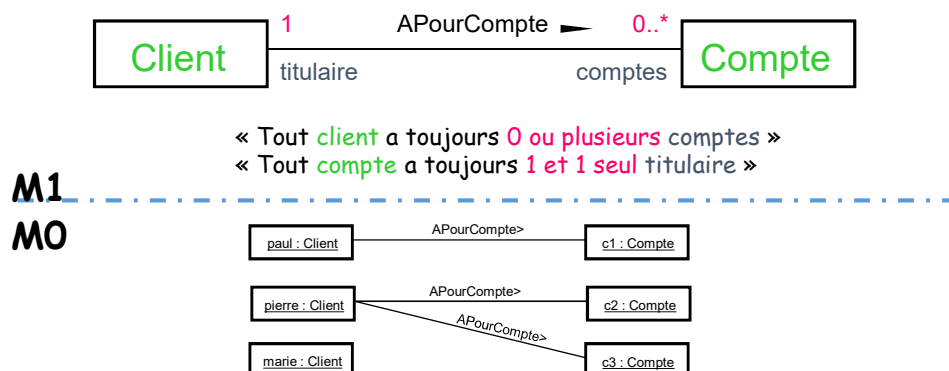
Utiliser les rôles pour «naviguer»



47

Cardinalités d'une association

- Précise combien d'objets peuvent être liés à un seul objet source
- Cardinalité minimale et cardinalité maximale ($C_{min}..C_{max}$)
- Doivent être des constantes



48

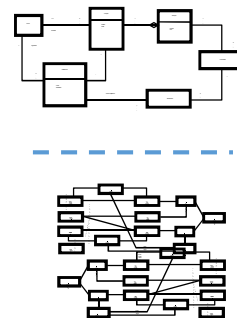
Cardinalité d'une association

— 1	Classe	Exactement une
— *	Classe	Plusieurs (0 à n), non ordonnés
{ordered} *	Classe	Plusieurs (0 à n), ordonnés
— 0,1	Classe	Optionnelle (0 ou 1)
— 1,2,4	Classe	Cardinalité spécifiée
— 1-10	Classe	Intervalle

49

Diagrammes de classes vs. d'objets

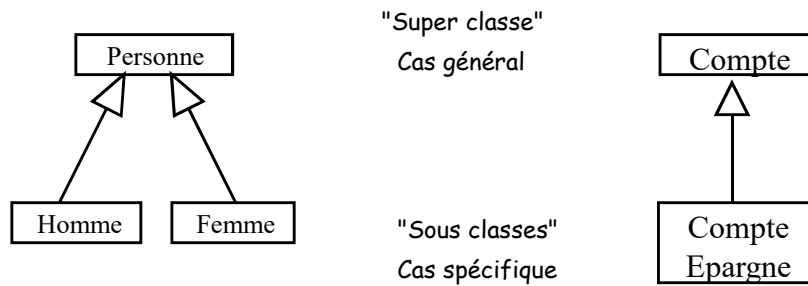
- Un diagramme de classes
 - définit l'ensemble de tous les états possibles
 - les contraintes doivent toujours être vérifiées
- Un diagramme d'objets
 - décrit un état possible à un instant t, un cas particulier
 - doit être conforme au modèle de classes
- Les diagrammes d'objets peuvent être utilisés pour
 - expliquer un diagramme de classe (donner un exemple)
 - valider un diagramme de classe (le "tester")



50

Généralisation / Spécialisation

Une classe peut être la généralisation d'une ou plusieurs autres classes.
Ces classes sont alors des spécialisations de cette classe.



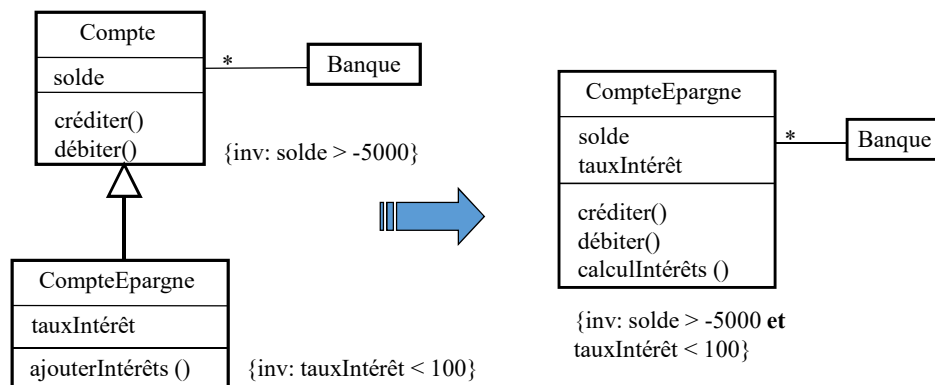
Deux points de vue liés (en UML) :

- relation d'héritage
- relation de sous-typage

51

Relation d'héritage

Les sous-classes « héritent » des propriétés des super-classes
(attributs, méthodes, associations, contraintes)

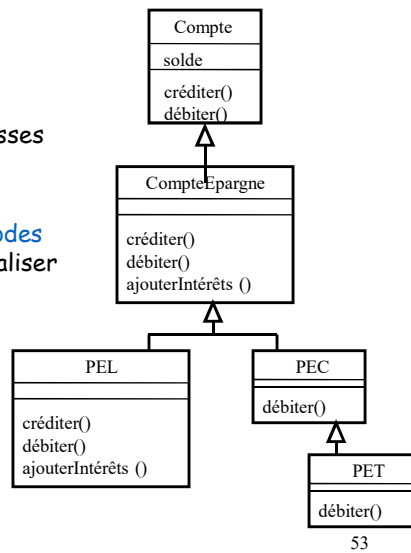


52

Relation d'héritage et redéfinitions

Une **opération** peut être "redéfinie" dans les sous-classes

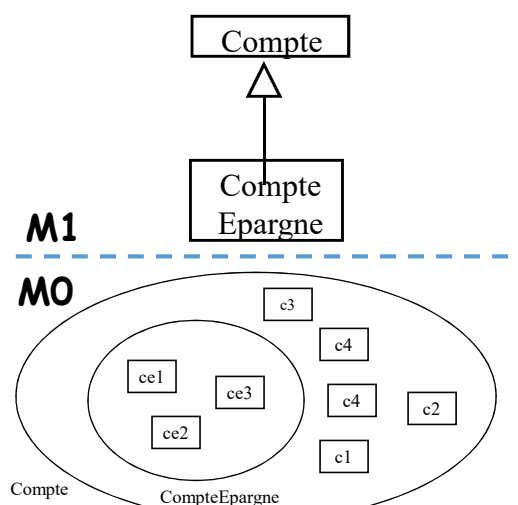
Permet d'associer des **méthodes** spécifiques à chaque pour réaliser une même opération



53

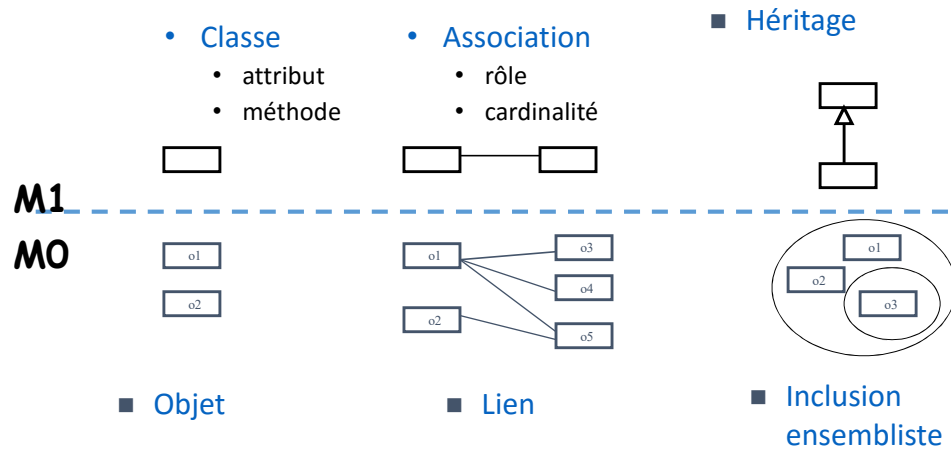
Relation de sous typage, Vision ensembliste

tout objet d'une sous-classe appartient également à la superclasse



54

Synthèse des concepts de base



55

Classes et Objets

Utilisation avancée

56

UML ? ... une question de style et de contexte

S'adapter ...

- au niveau d'abstraction
- au domaine d'application
- aux outils utilisés
- aux savants et ignorants
- ingénierie vs. retro-ingénierie

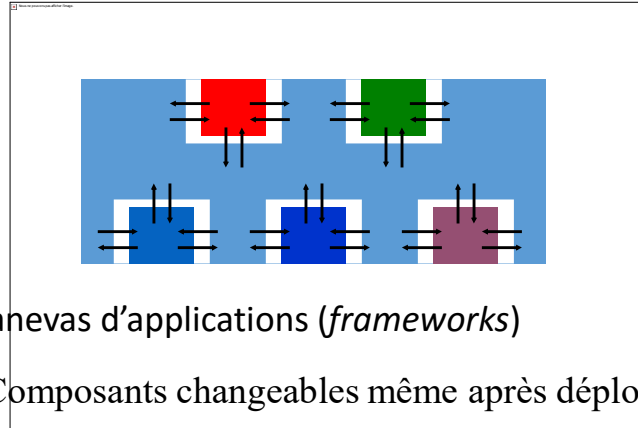
57

Principe de l'approche objet

- Structurer les systèmes autour des objets
 - Plutôt qu'autour des fonctions
- L'approche par modélisation facilite
 - Communication (et V&V)
 - avec donneur d'ordre (Validation)
 - entre les activités de dvp et de maintenance (Vérification)
 - Continuité entre les différentes phases du cycle de vie
 - cf. Jackson et JSD
- Obtenir des systèmes modulaires et maintenables
 - Notion de *lignes de produits*
 - Assemblage de briques de base vs. dvp ad-hoc
 - Produire des *canevas d'application* vs. un programme

58

Approche OO : Modélisation et Composants



- Canevas d'applications (*frameworks*)
- Composants changeables même après déploiement
- Garanties ?
Fonctionnelles , synchronisation, performances, QdS

59

De la difficulté de la validation intra-composant

```
Acquérir une valeur positive n
Tant que n > 1 faire
    si n est pair
        alors n := n / 2
    sinon n := 3n+1
Sonner alarme;
```

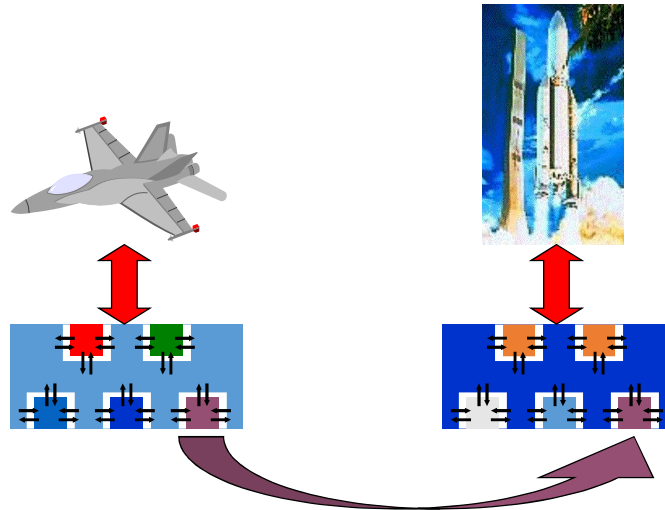
- Prouver que l'alarme est sonnée pour tout n?
- Indécidabilité de certaines propriétés
 - problème de l'arrêt de la machine de Turing...

■ Recours au test

- ici, si machine 32 bits, $2^{31} = 10^{10}$ cas de tests
- **5 lignes de code** => **10 milliards de tests** !

65

Validité inter-composants :
Peut-on (ré)-utiliser un composant?



66

Ariane 501 Vol de qualification Kourou,
ELA3 -- 4 Juin 1996, 12:34 UT

- H0 -> H0+37s : nominal
- Dans SRI 2:
 - BH (Bias Horizontal) > 2^15
 - convert_double_to_int(BH) fails!
 - exception SRI -> crash SRI2 & 1
- OBC disoriented
 - Angle attaque > 20°,
 - charges aérodynamiques élevées
 - Séparation des boosters



67

Ariane 501 : Vol de qualification

Kourou, ELA3 -- 4 Juin 1996, 12:34 UT

- H0 + 39s: auto-destruction (coût: 500M€)



68

Pourquoi ? (cf. *IEEE Comp.* 01/97)

- Pas une erreur de programmation
 - Non-protection de la conversion = décision de conception ~1980
- Pas une erreur de conception
 - Décision justifiée vs. trajectoire Ariane 4 et contraintes TR
- Problème au niveau du test d'intégration
 - Comme toujours, aurait pu être détecté. Mais gigantesque espace de test vs. ressources limitées
 - En plus, SRI inutile à cette étape du vol!

69

Pourquoi? (cf. *IEEE Computer* 01/97)

- Réutilisation dans Ariane 5 d'un composant de Ariane 4 ayant une contrainte « cachée » !
 - Restriction du domaine de définition
 - Précondition : $\text{abs}(\text{BH}) < 32768.0$
 - Valide pour Ariane 4, mais pas pour Ariane 5

70

Spécification = contrat entre un composant et ses clients

- Dans la vie réelle, différents types de contrats
 - Du « Contrat social » de Jean-Jacques Rousseau au “cash & carry”
- De même, plusieurs types de contrats dans un monde réparti



71

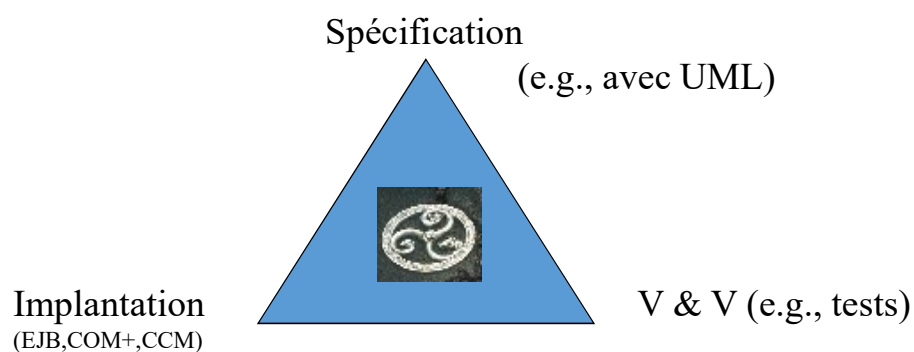
Quatre niveaux de contrats logiciels

- Élémentaire (syntaxique)
 - le programme compile...
- Comportemental (fonctionnel)
 - pré et post conditions
- Synchronisations
 - e.g. *path expressions*, etc. [McHale]
- Qualité de service (quantitative)
 - Négociation dynamique possible

Cf. *IEEE Computer*
July 1999

72

Composant de confiance?



Confiance = cohérence entre ces 3 aspects

73

Représentation des contrats en UML

- Typage par signature des méthodes insuffisant
 - besoin de pouvoir exprimer des restrictions
 - valeurs d'entrées et de sorties
 - besoin de préciser la sémantique
 - ce que fait une méthode (le quoi) sans entrer dans le détail comment : Indépendance vis-à-vis de l'implantation
- Inspirée par la notion de Type Abstrait de Données: Spécification = Signature +
 - Préconditions (conditions sous lesquelles une routine peut être appelée)
 - Postconditions (propriétés garanties par une routine)
 - Invariants de classe (vrai à l'entrée et à la sortie des routines)

74

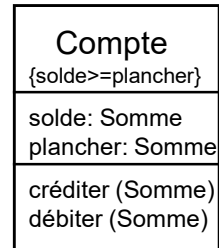
OCL : Object Constraint Language

- Langage de description de contraintes de UML
 - des contraintes restreignant les domaines de valeurs peuvent être ajoutées aux éléments du modèle UML
- Contrainte = expression booléenne (sans effet de bord) portant sur
 - opérations usuelles sur types de base (Boolean, Integer...)
 - attributs d'instances et de classes
 - opérations de « query » (fonctions sans effet de bord)
 - associations du modèle UML
 - états des StateCharts associés

75

Représentation des contraintes OCL

- Directement dans le modèle
 - notation entre { } **accrochée à un élément de modèle**



- Dans un document séparé, en précisant le **contexte**
 - Invariants = Propriétés vraies pour l'ensemble des instances de la classe
 - dans un état stable, chaque instance doit vérifier les invariants de sa classe

context Compte inv:
solde >= plancher

76

Précondition: *ce qui doit être respecté par le client*

- Spécification des conditions nécessaires pour qu'un client soit autorisé à appeler une méthode
 - exemple: montant > 0
- Notation en UML
 - {«precondition» OCL boolean expression}
 - Abbreviation: {pre: OCL boolean expression}

77

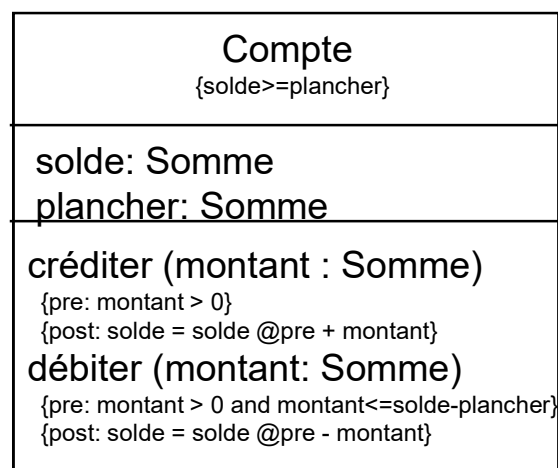
Postcondition:

Ce qui doit être assué par l'implantation

- Spécification de ce qui sera vrai à la complétion d'un appel valide à une méthode
 - exemple: `solde = solde @pre + montant`
- Notation en UML
 - {«postcondition» *OCL boolean expression*}
 - Abbreviation: {post: *OCL boolean expression*}
 - Opérateur pour accéder à la valeur « d'avant » (idem *old Eiffel*):
 - *OCL expression @pre*

78

Etre abstrait et précis avec UML



Analyse précise ou “analyse par contrat”

79

Conception par Contrat

- Le contraire de la *Programmation Défensive*
 - Essayer de tester tout ce qui pourrait poser problème
 - code lourd et complexe à maintenir et tester
 - quoi faire lorsqu'on détecte un problème?
- Assertions des contrats :
 - jouent un rôle crucial dans la séparation nette des responsabilités dans un système modulaire
 - contrat entre l'appelant d'une routine (le client) et l'implantation de la routine (le contractant) :

Pourvu que le client appelle la routine dans des conditions où l'invariant de classe du contractant et la précondition de la routine sont respectés, alors le contractant promet que lorsque la routine terminera, le travail spécifié dans la postcondition sera effectué, et l'invariant de classe sera respecté.

80

Intérêt pratique des contrats

- Specification, documentation
 - *Not a software fault tolerance gadget*
 - *Might help system fault tolerance...*
- Help V&V
 - When assertions are monitored
 - Never doing debugging again
- Help allocate responsibilities during integration
 - No longer have to find a scapegoat ;-)

81

Contract Violations: Preconditions

- The client broke the contract.
 - The provider does not have to fulfill its part of the contract.
 - If contracts are monitored, an exception should be raised
 - making it easy to identify the exact origin of the fault.

```

Unhandled exception: Routine failure. Exiting program.
Exception history:
=====
Object Routine
Type of exception      Description      Line
=====
#<BANK_ACCOUNT5f0c0>
precondition violated  positive_amount 63
=====
#<USER 5f000>
Routine failure      90
=====
#<DRIVER 5f010>
Routine failure      18
=====

```

82

Contract violations: Postconditions

- The implementation of a method did not comply with its promise:
This is a bug

```

Unhandled exception: Routine failure. Exiting program.
Exception history:
=====
Object Routine
Type of exception      Description      Line
=====
#<BANK_ACCOUNT5f0c0>
postcondition violated  deposited       70
=====
#<USER 5f000>
Routine failure      90
=====
#<DRIVER 5f010>
Routine failure      18
=====

```

- Again, easy to trace...(between lines 63-70)

83

Attributs dérivés

- Attributs dont la valeur peut être déduite d'autres éléments du modèle
 - e.g. *âge* si l'on connaît la date de naissance
 - notation : /age
- En termes d'analyse, indique seulement une *contrainte* entre valeurs et non une indication de ce qui doit être calculé et ce qui doit être mémorisé

84

+ # - Visibilité des éléments

- Restreindre l'accès aux éléments d'un modèle
- Contrôler et éviter les dépendances entre classes et paquetages

+	public	visible
#	protégé	visible dans la classe et ses sous-classes
-	privé	visible dans la classe uniquement
~	package	visible dans la package uniquement
- Utile lors de la conception et de l'implémentation, pas avant !
- N'a pas de sens dans un modèle conceptuel (abstrait)
- N'utiliser que lorsque nécessaire
- La sémantique exacte dépend du langage de programmation !

85

Déclaration d'attributs

[/] [visibilité] nom [: type] [card ordre] [= valeur-initiale] [{ props... }]

exemples:

```
age
+age
/age
- solde : Integer = 0
# age : Integer [0..1]
# numsecu : Integer {frozen}
# motsClés : String [*] {addOnly}
nbPersonne : Integer
```

- Adapter le niveau de détail au niveau d'abstraction

86

Déclaration d'opérations

[/] [visibilité] nom [(params)] [: type] [{ props... }]
params := [in / out / inout] nom [: type] [=default] [{
props... }]

```
/getAge()
+ getAge() : Integer
- updateAge( in date : Date ) : Boolean
# getName() : String [0..1]
+getAge() : Integer {isQuery}
+addProject() : { concurrency = sequential }
+addProject() : { concurrency = concurrent }
+main( in args : String [*] {ordered} )
```

- Adapter le niveau de détail au niveau d'abstraction

87

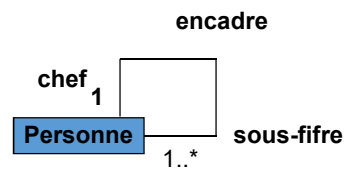
Raffinement du concept d'association

- Association uni/bi directionnelle
- Composition
- Aggrégation
- {frozen}, {addonly}, {ordered}, {nonunique}
- Classes associatives
- Associations qualifiées

88

Cas particuliers de relations

- Relations réflexives



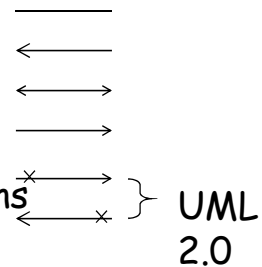
Une relation réflexive lie des objets de même classe

89

Navigation

Association unidirectionnelle

On ne peut naviguer que dans un sens



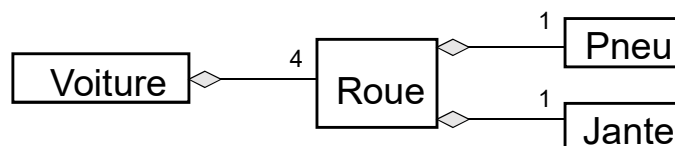
A priori, que dans les diagrammes de spécifications et d'implémentation

En cas de doute, **ne pas mettre de flèche !!!**

90

Composition

Notion intuitive de "composants" et de "composites"



composition =

cas particulier d'association

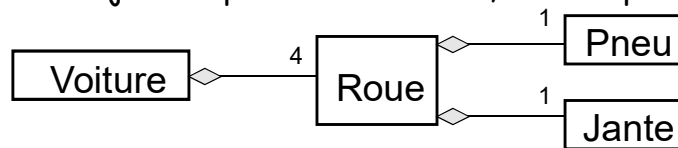
+ contraintes décrivant la notion de "composant"...

91

Composition

Contraintes liées à la composition :

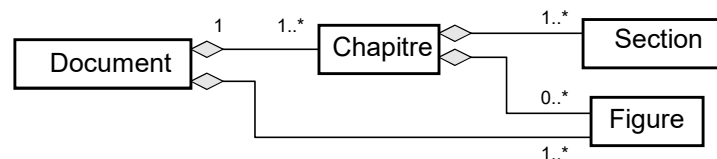
1. Un objet *composant* ne peut être que dans 1 seul objet *composite*
2. Un objet *composant* n'existe pas sans son objet *composite*
3. Si un objet composite est détruit, ses composants aussi



Dépend de la situation modélisée !
(Ex: vente de voitures vs. casse)

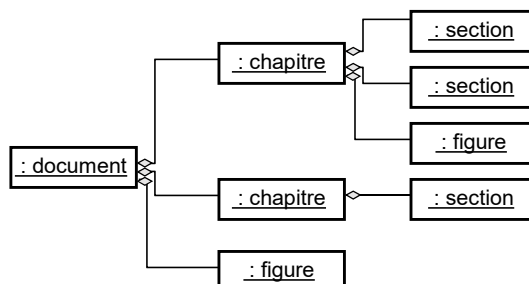
92

Composition



M1

M0

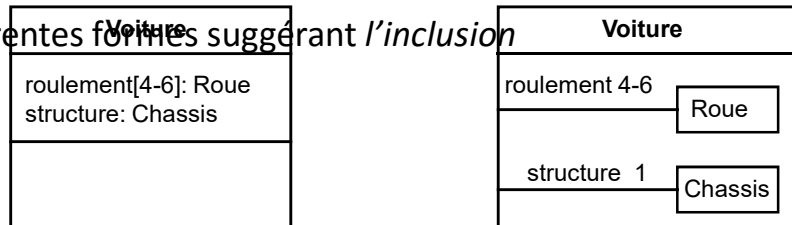


Contrainte :
le graphe d'objets
forme un arbre
(ou une forêt)

93

Autre vues de la composition/agrégation

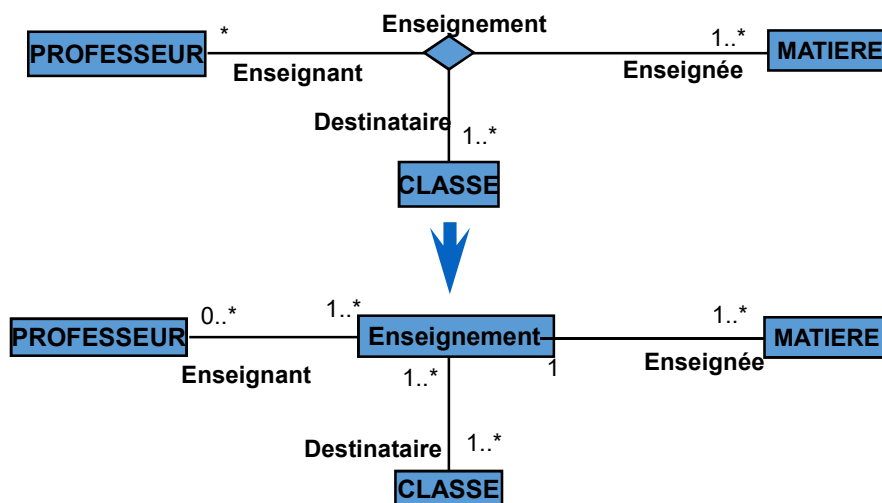
- Différentes formes suggérant l'inclusion



94

Relations n-aires

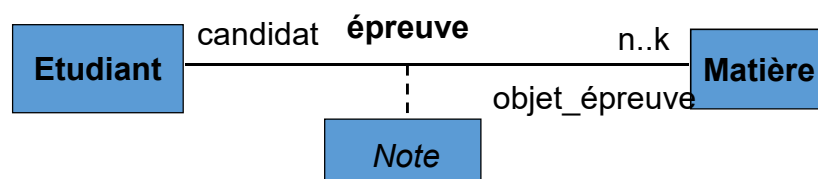
- Relations entre plus de 2 classes (*à éviter si possible*)



95

Associations attribuées

- L'attribut porte sur le lien

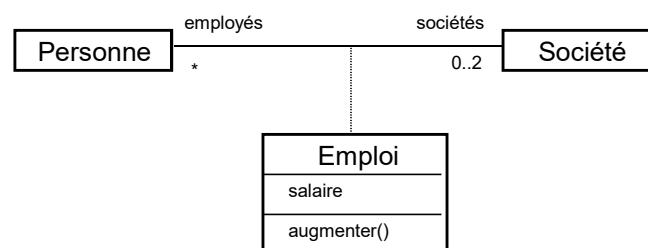


96

Classes association

Pour associer des attributs et/ou des méthodes aux associations

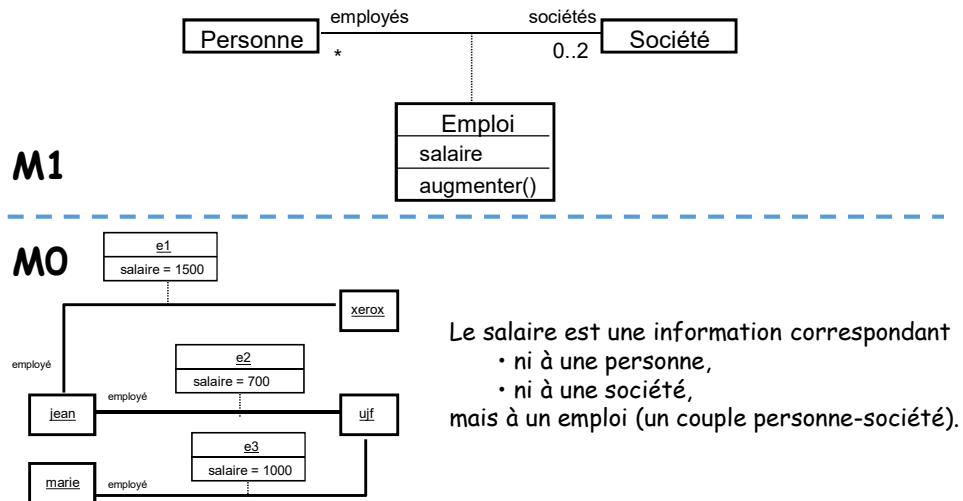
=> **classes associations**



Le nom de la classe correspond au nom de l'association
(problème: il faut choisir entre forme nominale et forme verbale)

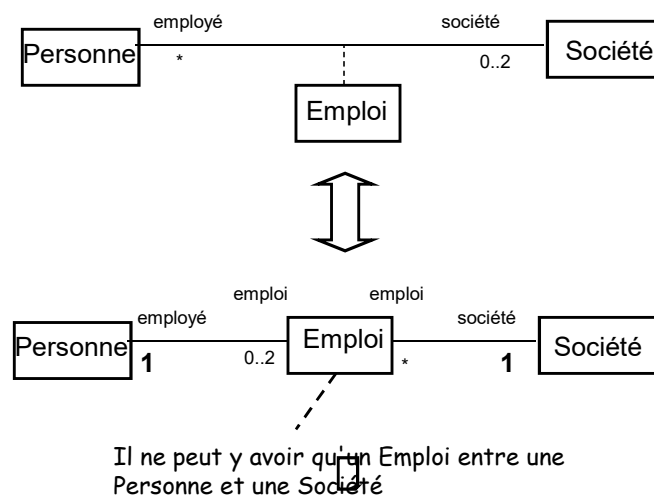
97

Classes association



98

Classes association : traduction



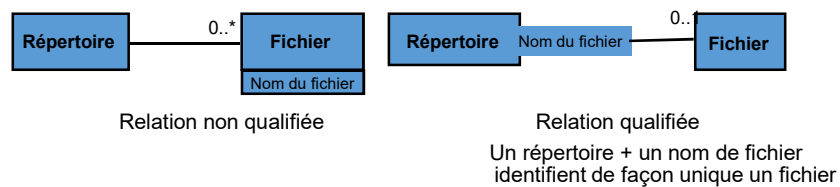
99

Qualifieurs d'associations

- Un qualifieur est un attribut spécial qui permet, dans le cas d'une relation 1-vers-plusieurs ou plusieurs-vers-plusieurs, de réduire la cardinalité. Il peut être vu comme une clé qui permet de distinguer de façon unique un objet parmi plusieurs.

100

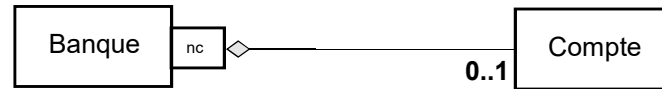
Qualifieurs d'associations : Exemple



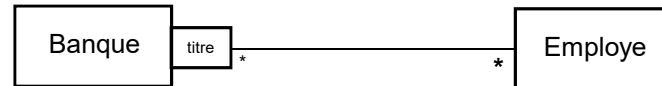
101

Cardinalité des Associations Qualifiées

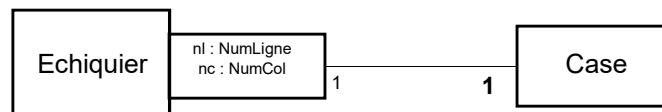
Cas classique: cardinalité 0..1



Cas plus rare: cardinalité * (pas de contrainte particulière exprimée)



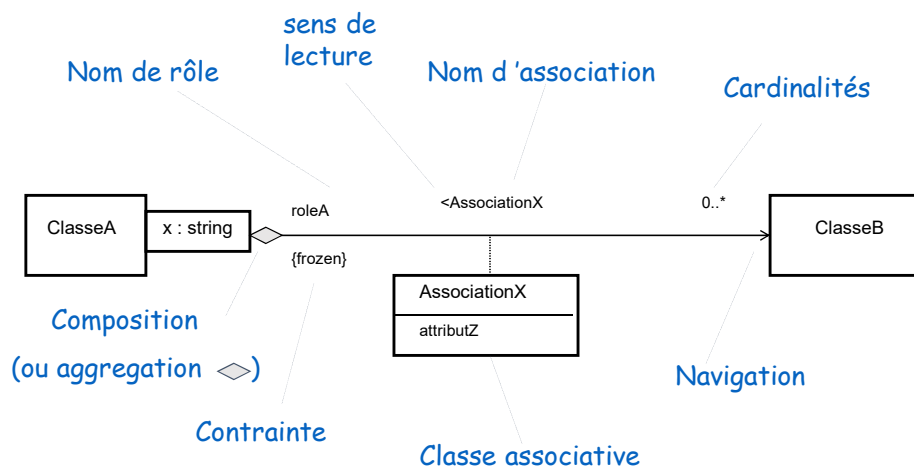
Cas plus rare: cardinalité 1 (généralement c'est une erreur)



0 comme cardinalité minimale, sauf si le domaine de l'attribut qualifieur est fini et toutes les valeurs ont une image.

102

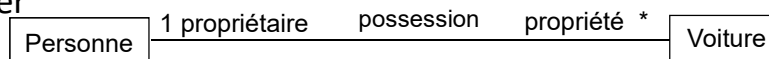
Synthèse sur les associations



103

Contraintes OCL navigant les relations

- Chaque association est un chemin de navigation
- Le contexte d'une expression OCL est le point de départ (la classe de départ)
- Les noms de rôles sont utilisés pour identifier quelle relation on veut naviguer



Context Voiture inv:
self.propriétaire.age >= 18

104

Navigation des relations 0..*

- Par navigation on n'obtient plus un scalaire, mais une *collection* d'objets
- OCL définit 3 sous-types de collections
 - **Set** : obtenu par navigation d'une relation 0..*
 - Context *Personne* inv: *propriété* retourne un Set[Voiture]
 - chaque élément est présent au plus une fois
 - **Bag** : si plus d'un pas de navigation
 - un élément peut être présent plus d'une fois
 - **Sequence** : navigation d'une relation {ordered}
 - c'est un Bag ordonné
- Nombreuses opérations prédéfinies sur les types *collection*.

Syntaxe :
Collection->opération

105

Opérations de base sur collections

- *isEmpty*
 - vrai si la collection n'a pas d'éléments
- *notEmpty*
 - vrai si la collection a au moins un élément
- *size*
 - nombre d'éléments dans la collection
- *count (elem)*
 - nombre d'occurrences de *elem* dans la collection

106

Opération *select*

- Syntaxes possibles
 - `collection->select(elem:T | expr)`
 - `collection->select(elem | expr)`
 - `collection->select(expr)`
- Sélectionne le sous-ensemble de *collection* pour lequel la propriété *expr* est vraie
- e.g.


```
context Personne inv:
  propriété->select(v: Voiture | v.kilometrage<100000)->notEmpty
```
- ou en raccourci :


```
context Personne inv:
  propriété->select(kilometrage<100000)->notEmpty
```

108

Opération *forAll*

- Syntaxes possibles
 - `collection->forall(elem:T | expr)`
 - `collection->forall(elem | expr)`
 - `collection->forall(expr)`
- Vrai si *expr* est vrai pour chaque élément de *collection*
- e.g.

```
context Personne inv:
  propriété->forall(v: Voiture | v.kilometrage<100000)
```

- ou en raccourci :

```
context Personne inv:
  propriété->forall(kilometrage<100000)
```

109

Autres opérations OCL

- `exists (expr)`
 - Vrai si *expr* est vrai pour au moins un élément de la collection
- `includes(elem), excludes(elem)`
 - vrai si *elem* est présent (resp. absent) dans la collection
- `includesAll(coll)`
 - vrai si tous les éléments de *coll* sont dans la collection
- `union (coll), intersection (coll)`
 - opérations classiques ensemblistes
- `asSet, asBag, asSequence`
 - conversions de type

110

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, *généricité*, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement



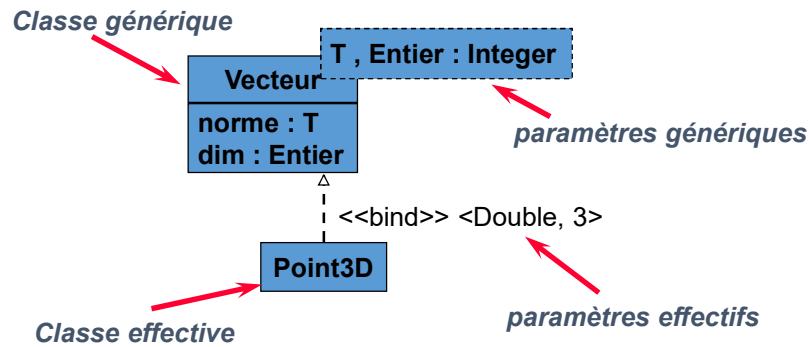
111

La généricité (Classes paramétrées)

- Indispensable pour les classes “conteneurs”
 - En Pascal, liste d'entiers, de réels, etc.
 - en C (C++) liste de (void *)
- Solution en Ada : donner un nom formel au type des éléments du conteneur (Liste[T])
 - Solution similaire en Eiffel (idem Ada) ou C++ (templates)
 - N'existe pas en Java = l'un de ses 3 points faibles principaux
 - Pas ce problème en Smalltalk (typage dynamique)
- Classes génériques vs. classes effectives

112

Représentation de la généricité



113

Modélisation UML

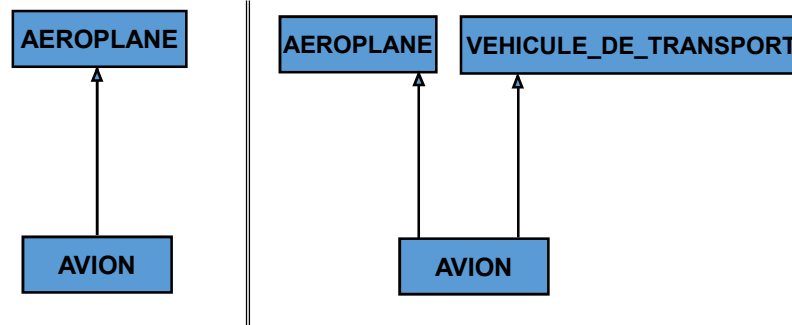
Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

114

Héritage (généralisation)

- Caractéristique des langages dit “orientés objets”
 - Partage d’attributs et d’opérations entre classes
 - Relation hiérarchique Super-classe/Sous-classe
 - Héritage multiple => graphe orienté (graphe d’héritage)



115

Utilisation de l’héritage

- Mécanisme d’extension de module
 - ajout de fonctionnalités dans la sous-classe
 - “customisation” et combinaison de composants logiciels
 - réutilisation de code
- Mécanisme de classification : sous-typage
 - relation X est-une-sort-de Y (est substituable à)
 - organisation des systèmes complexes (cf. Linnaeus)
 - réutilisation d’interface

116

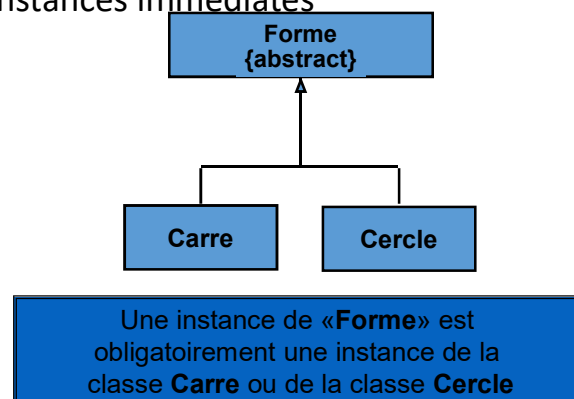
Classes abstraites

- Capturent des comportements communs
- Servent à structurer un système
- Ont des opérations dont l'implantation est absente
 - deferred en Eiffel
 - pure virtual en C++
 - abstract en Java
- Ne peuvent donc pas être instanciées
- Sémantique des opérations abstraites spécifiable
 - invariants, pre et post-conditions

117

Représentation de classes abstraites

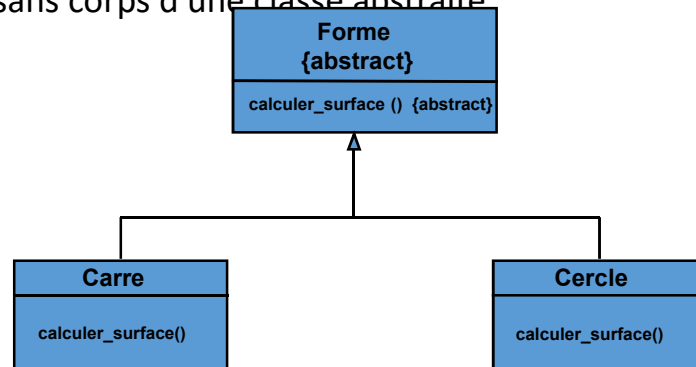
- Classes sans instances immédiates



118

Représentation des opérations abstraites

- Opération sans corps d'une classe abstraite



119

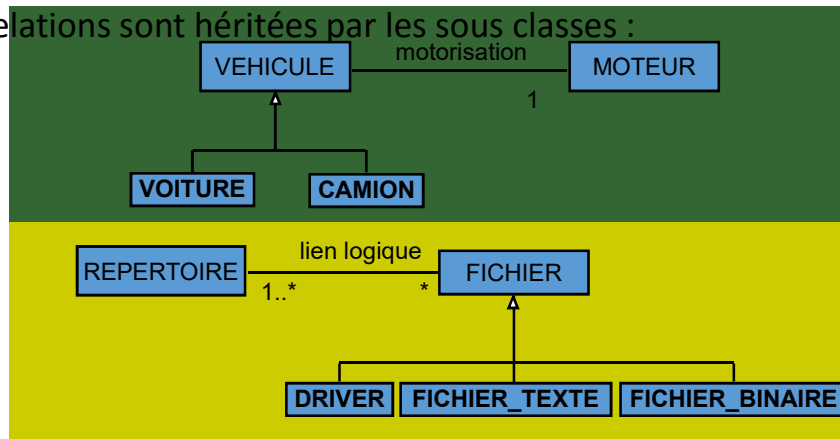
Interfaces et « lollipop »



120

Héritage des relations

- Les relations sont héritées par les sous classes :



121

Héritage des contrats

- LSP (Liskov Substituability Principle)
- Analogy with subcontracting
 - a routine subcontracts its implementation to its redefined version
- Preconditions can only be *weakened*
 - in Eiffel: **require else**
- Postconditions can only be strengthened
 - in Eiffel: **ensure then**

122

Polymorphisme et liaison dynamique

- Polymorphisme : possibilité de changer de forme
 - f : FORME; c : CERCLE; k : CARRE;
 - $f := c$; $f := k$
- Liaison dynamique : l'effet de l'appel d'une opération d'un objet dépend de sa forme effective à l'exécution
 - $f.imprimer$; -- *différent selon que f est CERCLE ou CARRE*
- Espace de nommage réduit et uniforme
 - Mise en facteur des parties communes
 - Possibilité de "conteneurs" hétérogènes

123

Polymorphisme : exemple

- T contient des FORMES
 - en fait des instances de sous-classes de FORME

- Programmes de type :

- $T[1] \leftarrow \text{carré}$
 - $T[2] \leftarrow \text{cercle}$

- ...

- Pour tout i
 $T[i].imprimer$

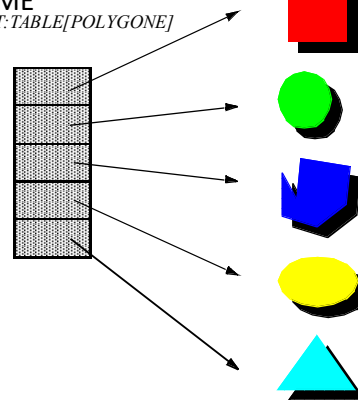
- Si ajouts ultérieurs:

- e.g. triangle

- Pas de modifications globales

- case-less programming

$T: \text{TABLE}[\text{POLYGONE}]$



124

Héritage et typage

- typage statique :
 - $x.f$ légal $\Leftrightarrow$ tout objet désigné par x “comprends” le message f
- liaison dynamique :
 - la bonne interprétation de f est choisie
- langages à objets :
 - Smalltalk : typage dynamique, liaison dynamique
 - C++ : typage (partiellement) statique, liaison dynamique pour les fonctions “virtuelles”
 - Eiffel, Java : typage statique, liaison dynamique
 - UML : au choix (!)

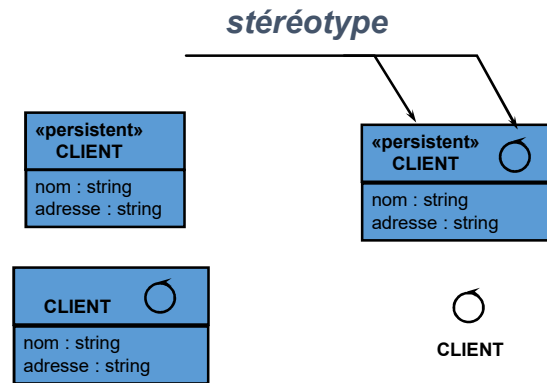
125

Les stéréotypes

- Nouveaux éléments de modélisation instanciant
 - Des classes du méta modèle UML (pour les stéréotypes de base UML)
 - Des extensions de classes du méta modèle UML (pour les stéréotypes définis par l'utilisateur)
- Peuvent être attachés aux éléments de modélisations et aux diagrammes :
 - Classes, objets, opérations, attributs, généralisations, relations, acteurs, uses-cases, événements, diagrammes de collaboration ...

126

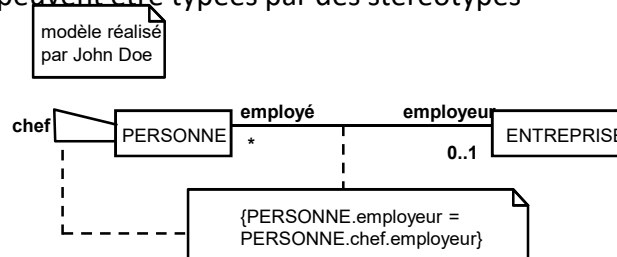
Notations pour les stéréotypes



127

Les notes

- Compléments de modélisation
 - Attachés à un élément du modèle ou libre dans un diagramme
 - Exprimés sous forme textuelle
 - Elles peuvent être typées par des stéréotypes



128

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- • Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

129

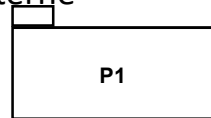
Notion de package

- Élément structurant les classes
 - Modularisation à l'échelle supérieure
 - Un package partitionne l'application :
 - Il référence ou se compose des classes de l'application
 - Il référence ou se compose d'autres packages
 - Un package régleme la visibilité des classes et des packages qu'il référence ou le compose
 - Les packages sont liés entre eux par des liens d'utilisation, de composition et de généralisation
 - Un package est la représentation informatique du contexte de définition d'une classe

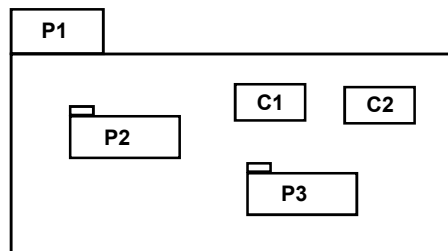
130

Représentation d'un package

- Vue graphique externe



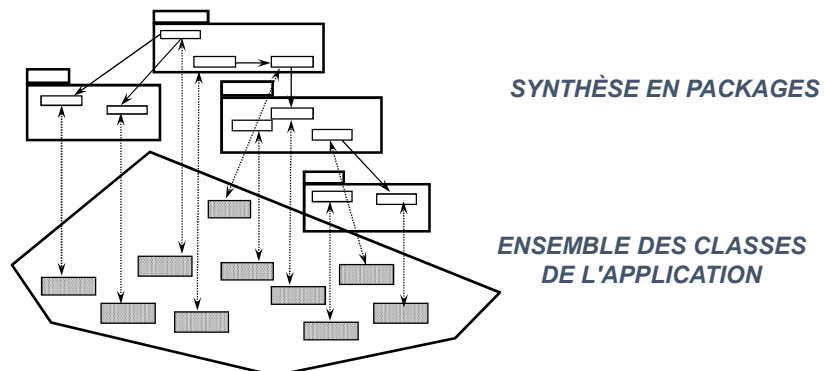
- Vue graphique externe et interne



131

Partitionnement d'une application

- Définition de vues partielles d'une application



N.B.: une classe appartient à un et un seul package

132

Visibilité dans un package

- Réglementation de la visibilité des classes
 - **Classes de visibilité publique :**
 - classes utilisables par des classes d'autres packages
 - **Classes de visibilité privée :**
 - classes utilisables seulement au sein d'un package
- Représentation graphique

{public }

CLASSE D'INTERFACE

Classe
{private }

CLASSE DE CORPS

Package::Classe

CLASSE EXTERNE

133

Utilisation entre packages

- Définition
 - Il y a utilisation entre packages si des classes du package utilisateur accèdent à des classes du package utilisé
 - Pour qu'une classe d'un package p1 puisse utiliser une classe d'un package p2, il doit y avoir au préalable une déclaration **explicite** de l'utilisation du package p2 par le package p1
- Représentation graphique

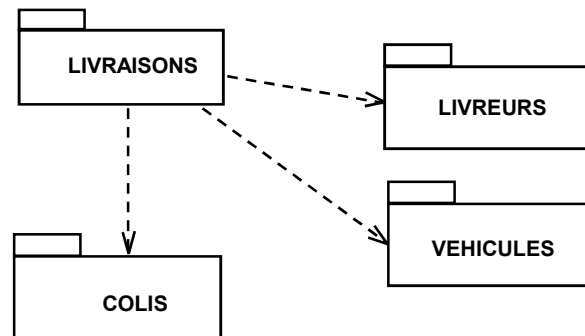


Vue externe du package P1

134

Utilisation entre packages

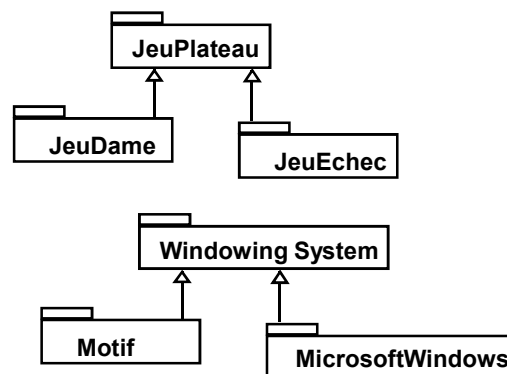
- Exemple (vue externe du package livraisons)



135

Héritage entre packages

- Exemples



136

Utilité des packages

- Réponses au besoin
 - Contexte de définition d'une classe
 - Unité de structuration
 - Unité d'encapsulation
 - Unité d'intégration
 - Unité de réutilisation
 - Unité de configuration
 - Unité de production

137

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- ➔ • Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OÙ?*)
 - Diagramme de composants et de déploiement

142

Expression des besoins et OOAD

- Sujet longtemps négligé (e.g. OMT)
 - dérive de certains projets vers le « conceptuel »
- Question de l'expression des besoins pourtant fondamentale
 - Et souvent pas si facile (*cible mouvante*)
 - cf. *syndrome de la balançoire*
- Object-Oriented Software Engineering (Ivar Jacobson et al.)
 - Principal apport : la technique des acteurs et des cas d'utilisation
 - Cette technique est intégrée à UML

143

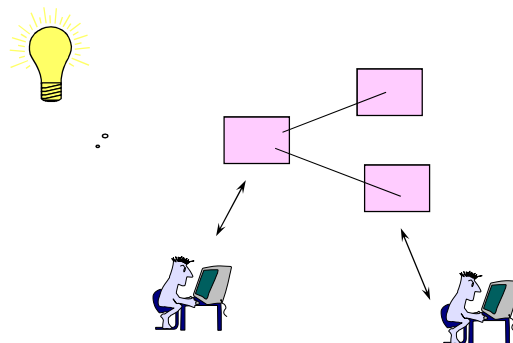
Quatre objectifs

☆ Se comprendre

🕒 Représenter le système

🕒 Exprimer le service rendu

🕒 Décrire la manière dont le système est perçu



144

Les moyens

- On imagine le système « fini »
- On montre comment on interagit avec lui
 - Les acteurs UML
 - Les *use-cases* UML

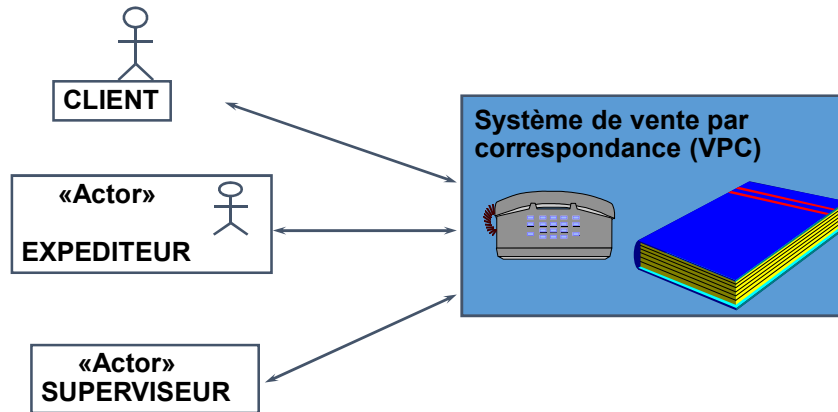
145

Acteurs

- Entité externe au système et amenée à interagir avec lui. Un acteur «joue un rôle» vis-a-vis du système
- Un acteur est une classe
- Un acteur peut représenter un être humain, un autre système, ...
- L'identification des acteurs permet de délimiter le système

148

Acteurs : notations



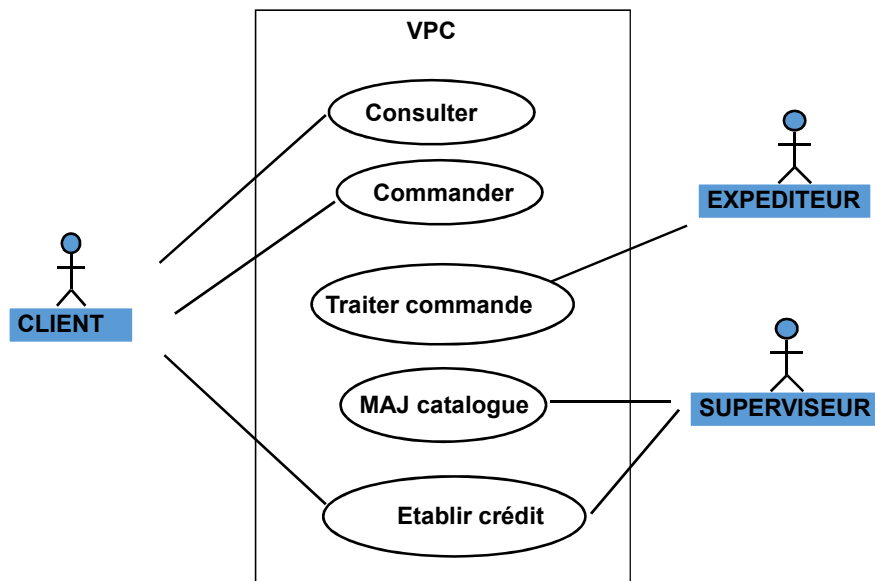
149

Les cas d'utilisation (use-cases)

- Un cas d'utilisation est une manière particulière d'utiliser le système
 - séquence d'interactions entre le système et un ou plusieurs acteurs
 - Ils s'expriment par des diagrammes de séquences
- La compilation des cas d'utilisation décrit de manière informelle le service rendu par le système
 - fournissent une expression "fonctionnelle" du besoin
 - peuvent piloter la progression d'un cycle en spirale
- Les cas d'utilisation sont nommés en utilisant la terminologie décrite dans le dictionnaire

150

Cas d'utilisation : exemple et notation



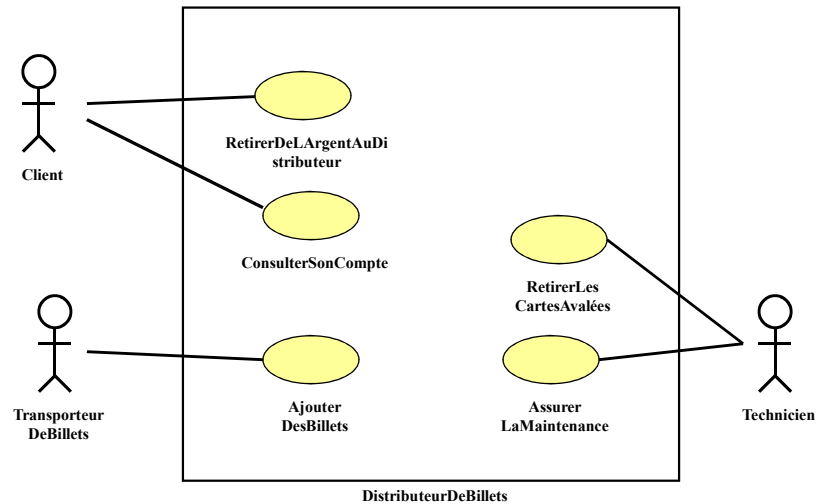
151

Exemple de cas d'utilisation



152

Diagramme de cas d'utilisation

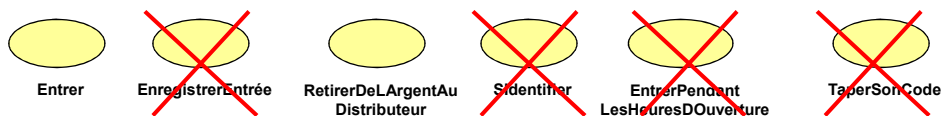


153

Cas d'utilisation (CU)

CasDUtilisationX

- Cas d'utilisation (CU)
 - une manière d'utiliser le système
 - une suite d'interactions entre un acteur et le système
- Correspond à une fonction du système **visible par l'acteur**
- Permet à un acteur d'atteindre **un but**
- Doit être **utile en soi**
- Regroupe un **ensemble de scénarii** correspondant à un même **but**



154



Description préliminaire des cas d'utilisation

- Pour chaque cas d'utilisation
 - ◆ choisir un identificateur représentatif
 - ◆ donner une **description textuelle simple**
 - ◆ la fonction réalisée doit être comprise de tous
 - ◆ préciser ce que fait le système, ce que fait l'acteur
 - ◆ pas trop de détails, se concentrer sur le **scénario "normal"**



Retirer
DeLArgent
AuDistributeur

Quand un client a besoin de liquide il peut en utilisant un distributeur retirer de l'argent de son compte. Pour cela :

- le client insère sa carte bancaire dans le distributeur
- le système demande le code pour l'identifier
- le client choisit le montant du retrait
- le système vérifie qu'il y a suffisamment d'argent
- si c'est le cas, le système distribue les billets et débite le compte du client
- le client prend les billets et retire sa carte

155

Limites du système et interfaces

Limites du système

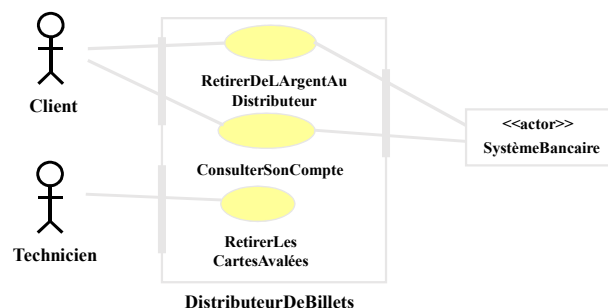


interface de communication

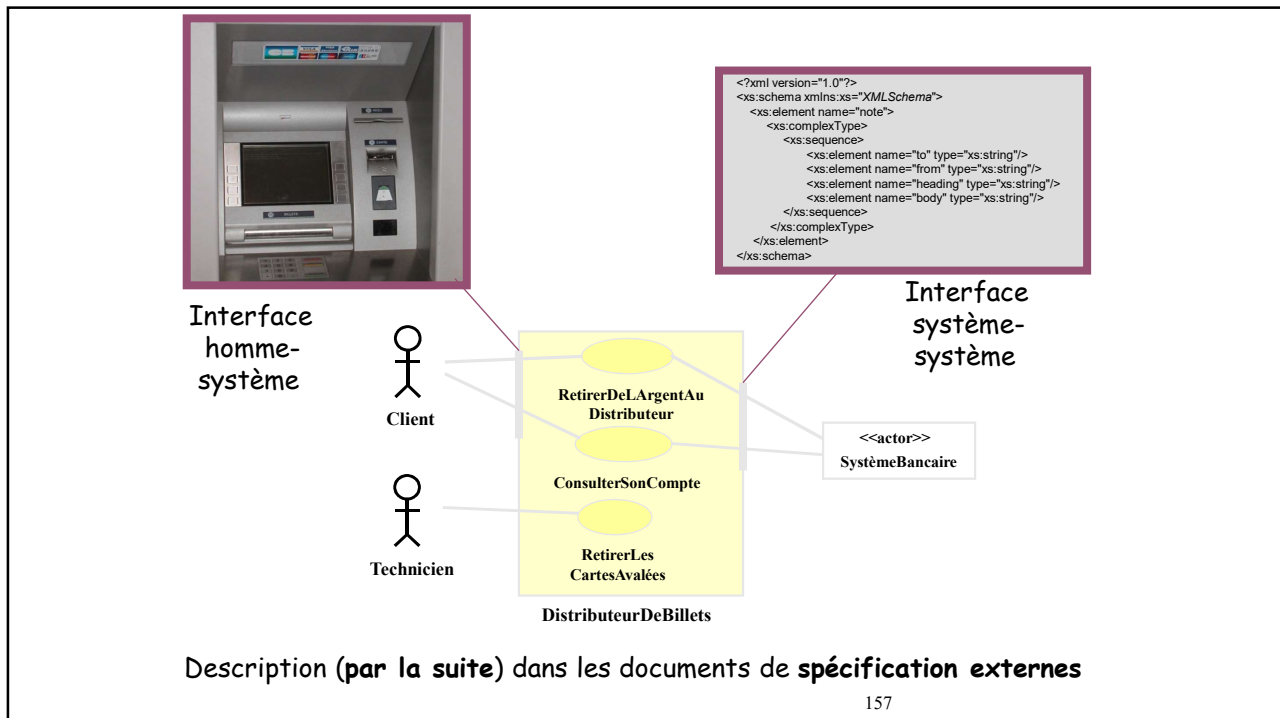
- acteur humain
- acteur logiciel

interface homme - système

interface logicielle (e.g. API)



156



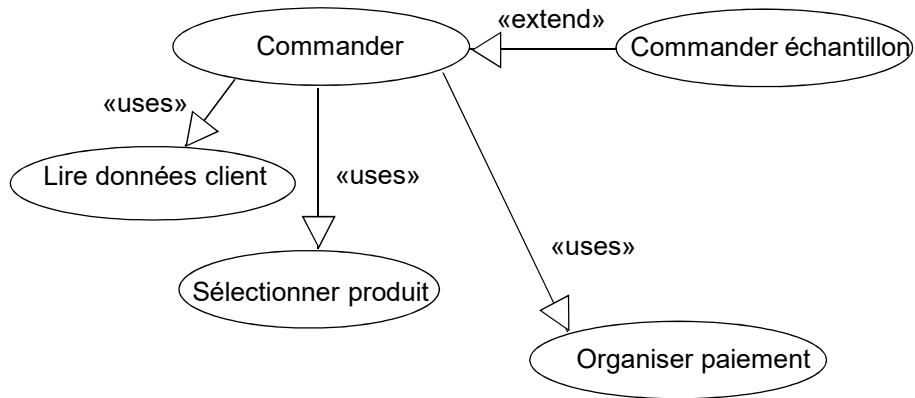
157

Relations sur les *use-cases*

- Communication
 - Lien entre le use case et l'acteur.
- Utilisation («*uses*»)
 - Utilisation d'autres use-cases pour en préciser la définition
- Extension («*extends*»)
 - Un use-case étendu est une spécialisation du use-case père

158

Relations sur les *use-cases* : notation



159

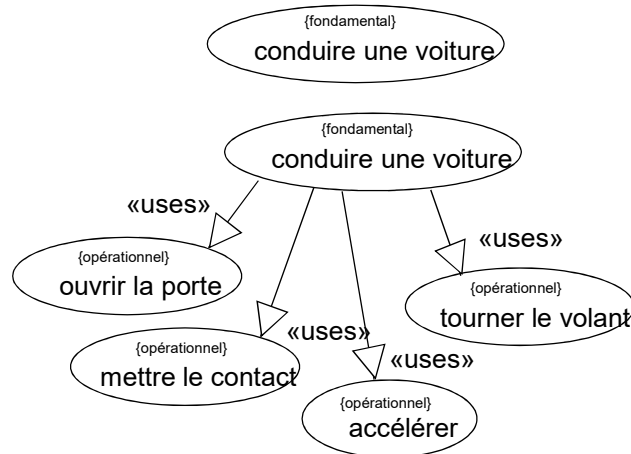
Exprimer le service rendu

- Besoins fondamentaux : manières d'utiliser le système
 - Représentation globale par cas d'utilisation
 - $\forall$ taille du système, seulement de 3 à 10 Use Cases
- Besoins opérationnels : interactions avec le système
 - Représentation détaillée par raffinement des cas d'utilisation
 - Début de décomposition fonctionnelle : ne pas aller trop loin

160

Besoins fondamentaux et opérationnels

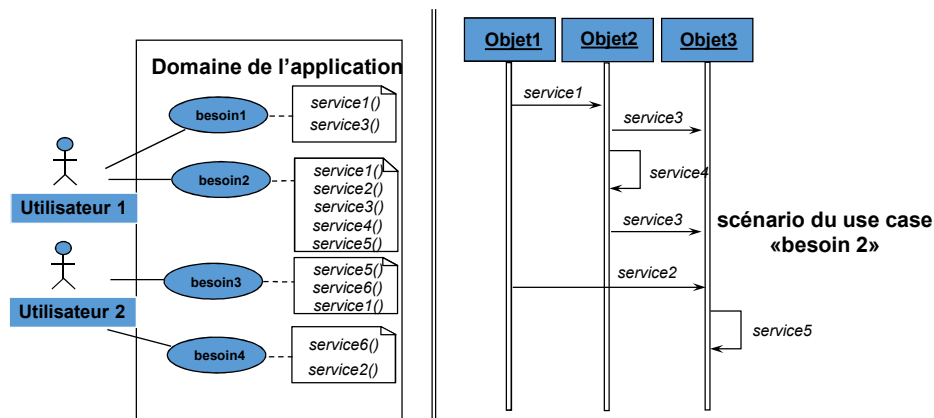
- **Besoin fondamental :**
 - Conduire une voiture
- **Besoins opérationnels**
 - Ouvrir la porte
 - Mettre le contact
 - Accélérer
 - Tourner le volant
 - ...



161

Utiles pour l'établissement de scénarios

- Modélisation d'exemples issus des use-cases



162

ATTENTION

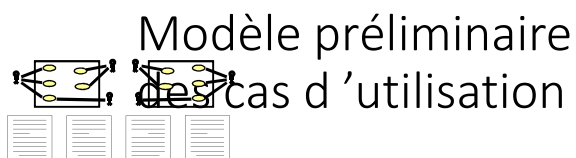
"Congratulations: Use Cases Have Been Written, and Are Imperfect"

[Applying UML and Patterns, Craig Larman]

"A big danger of use cases is that people make them too complicated and get stuck. Usually you'll get less hurt by doing too little than by doing too much".

[UML Distilled, Martin Fowler]

163



- Équivalent à définir une **table des matières** et des résumés pour chaque chapitre
- Pas de règles strictes
- Effectuer les meilleurs regroupement possibles
- Rester simple !
- Structuration possible en termes de paquetages
- Culture d'entreprise

Stabilisation du modèle par **consensus grandissant**

164

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement



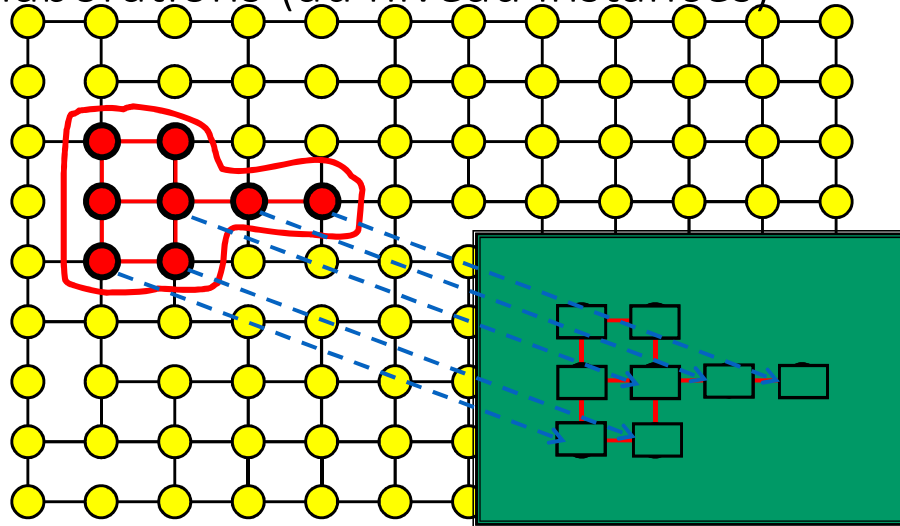
165

Aspects dynamiques du système

- Jusqu'ici, système décrit *statiquement*:
 - Décrivent les messages (méthodes ou opérations) que les instances des classes peuvent recevoir mais ne décrivent pas l'émission de ces messages
 - Ne montrent pas le lien entre ces échanges de messages et les processus généraux que l'application doit réaliser
- Il faut maintenant décrire comment le système évolue dans le temps
- On se focalise d'abord sur les **collaborations** entre objets. Rappel :
 - objets : simples
 - gestion complexité : par collaborations entre objets simples

166

Collaborations (au niveau instances)



167

Selon T. Reenskaug...

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement



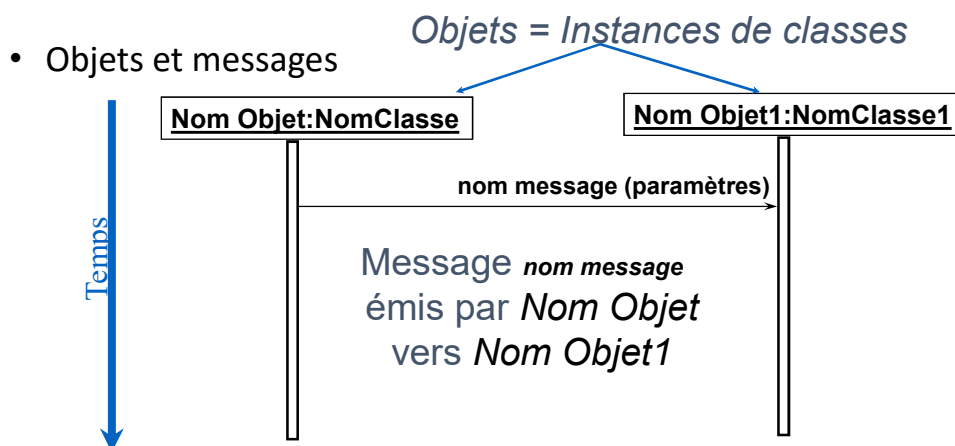
168

Diagrammes de séquences (scénarios)

- Dérivés des scénarios de OMT :
 - Montrent des exemples de coopération entre objets dans la réalisation de processus de l'application
 - Illustrent la dynamique d'enchaînement des traitements à travers les messages échangés entre objets
 - le temps est représenté comme une dimension explicite
 - en général de haut en bas
- Les éléments constitutifs d'un scénario sont :
 - Un ensemble d'objets (et/ou d'acteurs)
 - Un message initiateur du scénario
 - La chronologie des messages échangés subséquentement
 - Les contraintes de temps (aspects temps réel)

169

Syntaxe graphique



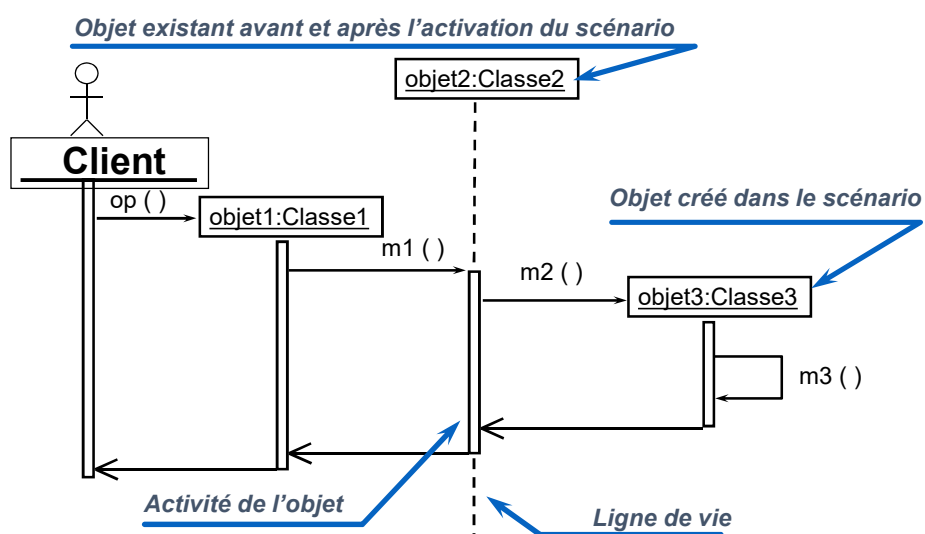
170

Ligne de vie et activation

- La «ligne de vie» représente l'existence de l'objet à un instant particulier
 - Commence avec la création de l'objet
 - Se termine avec la destruction de l'objet
- L'activation est la période durant laquelle l'objet exécute une action lui-même ou via une autre procédure

171

Notation



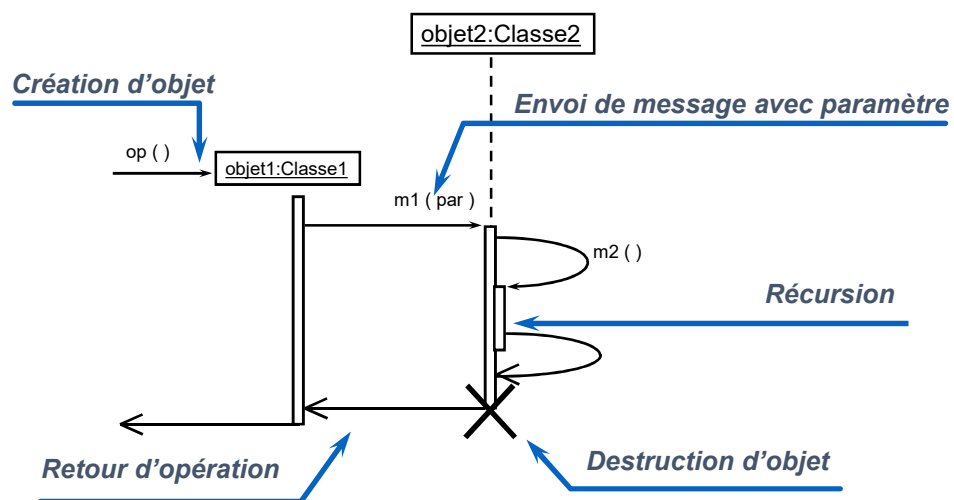
172

Messages

- Communication entre objets
 - Des paramètres
 - Un retour
- Cas particuliers
 - Les messages entraînant la construction d'un objet
 - La récursion
 - Les destructions d'objets

173

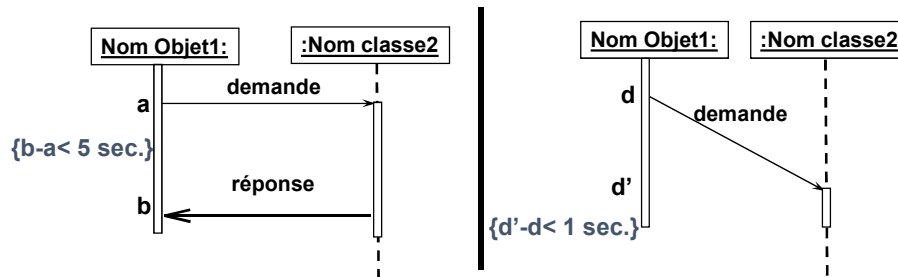
Notations



174

Aspects asynchrones et temps réel

- Lecture du scénario et chronologie
 - Un scénario se lit de haut en bas dans le sens chronologique d'échange des messages.
 - Des contraintes temporelles peuvent être ajoutées au scénario



175

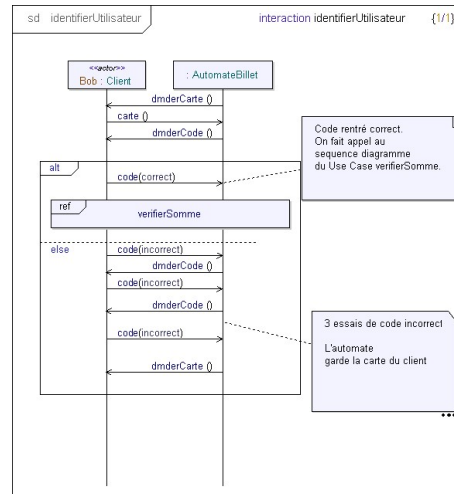
Fragments combinés (UML2)

- Les fragments combinés permettent de décrire des diagrammes de séquence de manière compacte.
- Pour combiner des fragments de séquences, il existe une dizaine d'opérateurs définis dans la notation UML 2.0.
 - **Alternative (alt)**
 - **Option (opt)**
 - **Boucle (loop)**
 - **Parallèle (par)**
 - « Break », pour indiquer que le reste du scénario ne sera pas couvert
 - « Neg », pour les scénarios d'abus ou invalides
 - « Critical », pour désigner une section critique
 - « Ignore » et « Consider », pour filtrer les message
 - « Assert », pour indiquer une assertion dans un scénario
 - Séquençage faible ou stricte

177

Fragments combinés: alternative

- Exemple: soit l'utilisateur entre un code correct et dans ce cas le diagramme de séquence relatif à la vérification du code est appelé, soit l'utilisateur entre un code erroné trois fois et sa carte est gardée.
- On remarque ici une référence (**ref**) vers un diagramme défini ailleurs.



178

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement



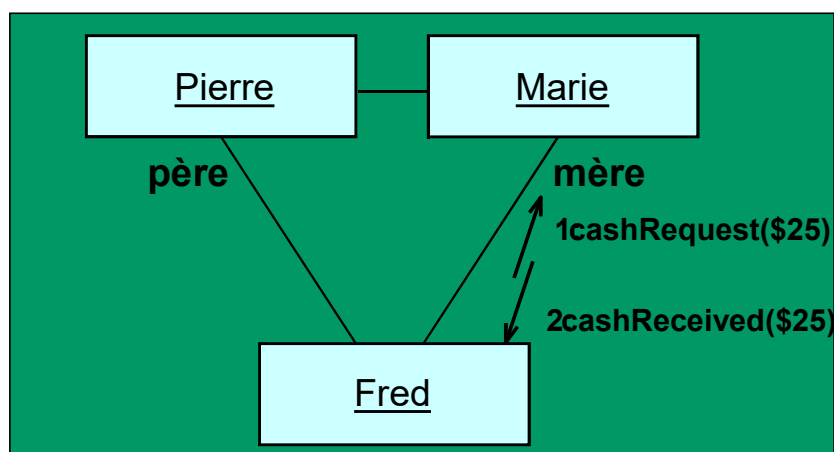
179

Diagrammes de collaboration

- Les scénarios et diagrammes de collaboration:
 - Montrent des exemples de coopération des objets dans la réalisation de processus de l'application
- Les scénarios :
 - Illustrent la dynamique d'enchaînement des traitements d'une application en introduisant la dimension temporelle
- Les diagrammes de collaboration
 - Dimension temporelle représentée par numéros de séquence : définition d'un ordre partiel sur les opérations
 - Représentation des objets et de leurs relations
 - Utilisent les attributs et opérations

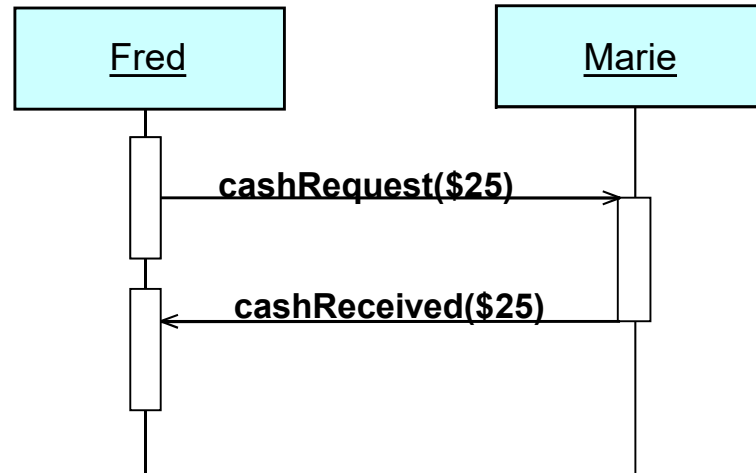
180

Représentation d'une collaboration (niveau instance)



181

Equivalent au diagramme de séquence:



182

Éléments constitutifs

- Un contexte contenant les éléments mis en jeu durant l'opération :
 - Un acteur
 - Un ensemble d'objets, d'attributs et de paramètres
 - Des relations entre ces objets
- Des interactions
 - Des messages
 - Un message initiateur du diagramme provenant d'un
 - Acteur de l'application,
 - Objet de l'application.
 - Les numéros de séquence des messages échangés entre les objets de cet ensemble suite au message initiateur

184

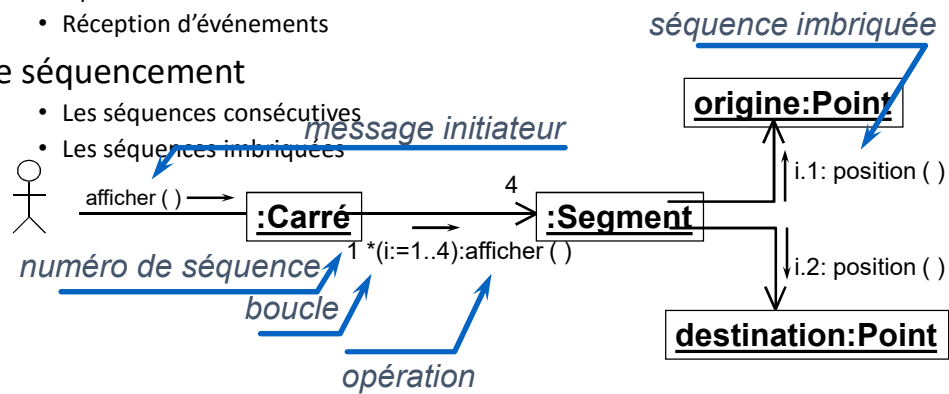
Syntaxe graphique

- Les messages

- Opérations
- Réception d'événements

- Le séquençement

- Les séquences consécutives
- Les séquences imbriquées



185

Questions auxquelles répondent les collaborations

- Quel est l'objectif ?
- Quels sont les objets ?
- Quelles sont leurs responsabilités ?
- Comment sont-ils interconnectés ?
- Comment interagissent-ils ?

186

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement



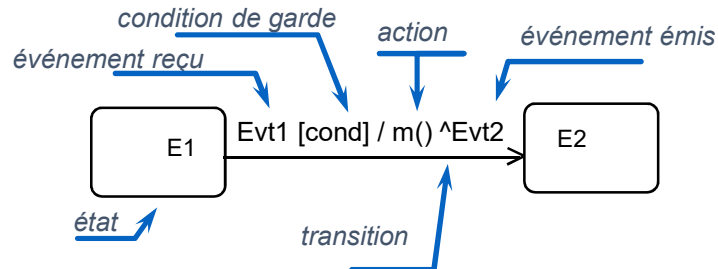
187

Les diagrammes d'états

- Attachés à une classe
 - Généralisation des scénarios
 - Description systématique des réactions d'un objet aux changements de son environnement
- Décrivent les séquences d'états d'un objet ou d'une opération :
 - En réponse aux «stimulis» reçus
 - En utilisant ses propres actions (transitions déclenchées)
- Réseau d'états et de transitions
 - Automates étendus
 - Essentiellement *Diagrammes de Harel (idem OMT)*

188

Syntaxe graphique: diagramme d'états



Syntaxe :

ÉvénementReçu (param : type, ...) [condition de garde] / Action ^ÉvénementsEmis

189

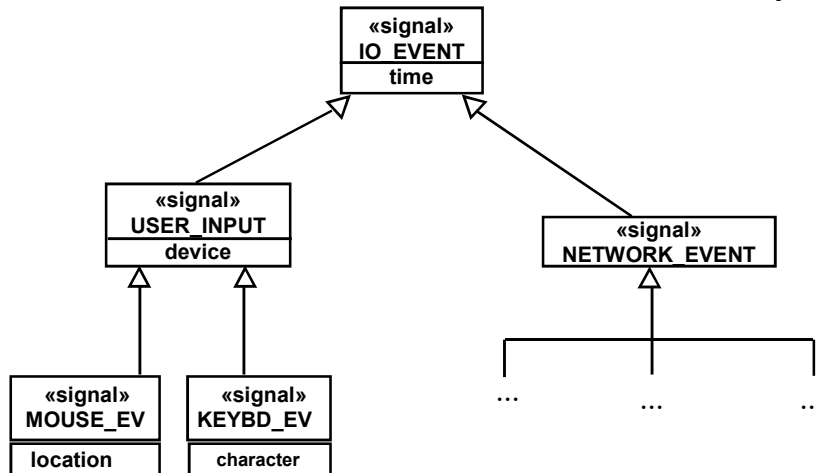
Notion d'événements

- Stimulis auxquels réagissent les objets
 - Occurrence déclenchant une transition d'état
- Abstraction d'une information instantanée échangée entre des objets et des acteurs
 - Un événement est instantané
 - Un événement correspond à une communication unidirectionnelle
 - Un objet peut réagir à certains événements lorsqu'il est dans certains états.
 - Un événement appartient à une *classe d'événements* (classe stéréotypée «signal»).

190

Les événements

- Les événements sont considérés comme des objets



191

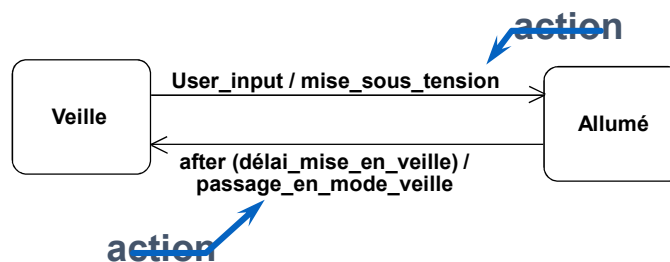
Typologie d'événements

- Réalisation d'une condition arbitraire
 - transcrit par une condition de garde sur la transition
- Réception d'un signal issu d'un autre objet
 - transcrit en un événement déclenchant sur la transition
- Réception d'un appel d'opération par un objet
 - transcrit comme un événement déclenchant sur la transition
- Période de temps écoulée
 - transcrit comme une expression du temps sur la transition
 - E.g. *after(5s)*

192

Notion d'action

- Action : opération *instantanée* (conceptuellement) et *atomique* (ne peut être interrompue)
- Déclenchée par un événement
 - Traitement associé à la transition
 - Ou à l'entrée dans un état ou à la sortie de cet état



193

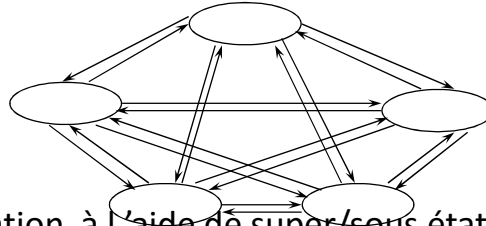
Notion d'états

- Etat : situation stable d'un objet parmi un ensemble de situations pré-définies
 - conditionne la réponse de l'objet à des événements
 - programmation réactive / « temps réel »
 - Intervalle entre 2 événements, il a une durée
- Peut avoir des variables internes
 - attributs de la classe supportant ce diagramme d'états

194

Structuration en sous-états

- Problème d'un diagramme d'états plats
 - Pouvoir d'expression réduit, inutilisable pour de grands problèmes
 - Explosion combinatoire des transitions.

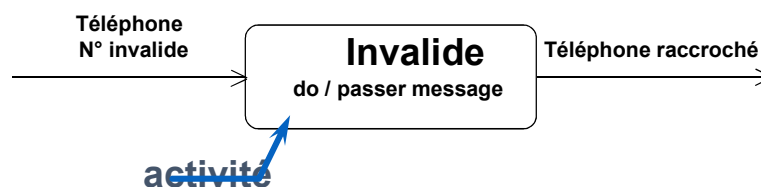


- Structuration à l'aide de super/sous états (+ hiérarchies d'événements)
 - représentés par imbrication graphique

195

Notion d'activité dans un état

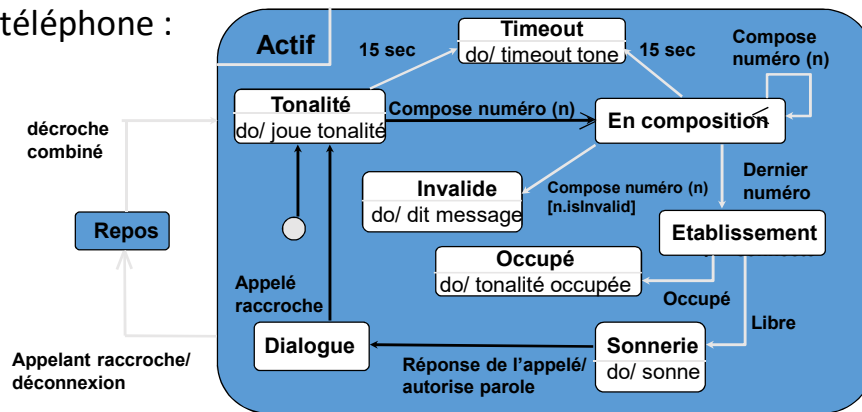
- Activité : opération se déroulant continuellement tant qu'on est dans l'état associé
 - *do / action*
- Une activité peut être interrompue par un événement.



196

Exemple de diagramme d'états

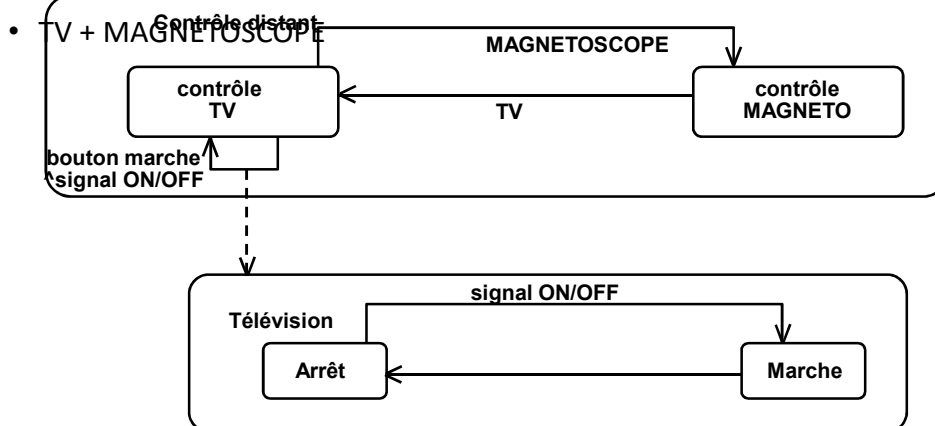
- un téléphone :



197

Émission d'événements

- Automate d'états d'une télécommande double :



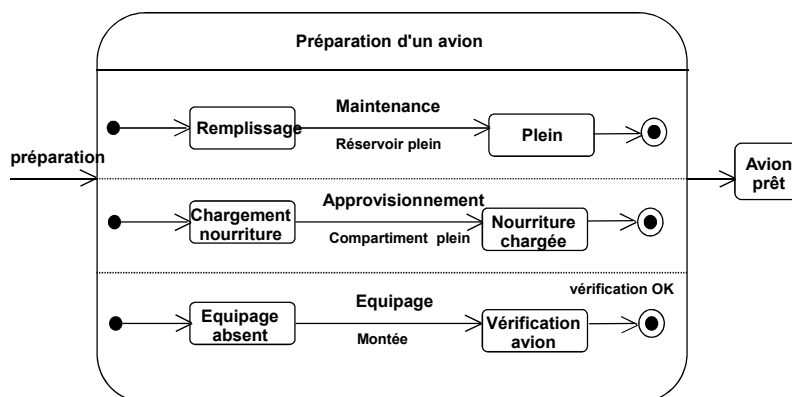
198

Diagrammes d'états concurrents

- Utilisation de sous-états concurrents pour ne pas à avoir à expliciter le produit cartésien d'automates
 - si 2 ou plus aspects de l'état d'un objet sont indépendants
 - Activités parallèles
- Sous-états concurrents séparés par pointillés
 - « *swim lanes* »

199

Exemple de concurrence



200

Etat-transition (résumé)

- Format :
 - événement (arguments) [conditions] / action ^événements provoqués
- Déclenchement :
 - par un événement (peut être nul).
 - Peut avoir des arguments.
 - Conditionné par des expressions booléennes sur l'objet courant, l'événement, ou d'autre objets.
- Tir de la transition :
 - Exécute certaines actions instantanément.
 - Provoque d'autres événements ; globaux ou vers des objets cibles.

201

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- ➔ • Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

202

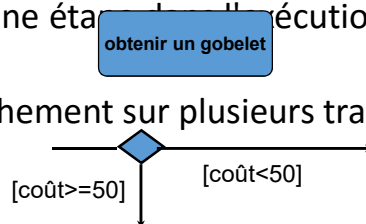
Les diagrammes d'activité

- Traitements effectués par une opération
 - Description d'un flot de contrôle procédural
 - Réseau d'actions et de transitions : automate dégénéré
 - La transition s'effectue lorsque l'opération est terminée
 - Pas de déclenchement par événement asynchrone
 - Sinon utilisation diagrammes d'états classiques
- Attachés à
 - une classe,
 - une opération,
 - ou un *use-case* (*workflow*)

203

Etat-action et décision

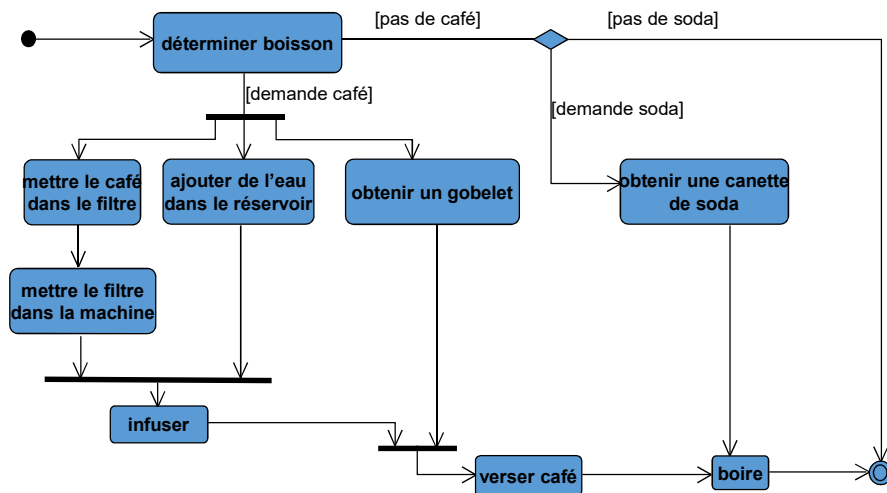
- Etat-action = raccourci pour un état où il y a :
 - une action interne
 - au moins une transition sortante
 - production d'un événement implicite : action accomplie
 - Pas de production/réaction à des événements explicites
- Modélisation d'une étape dans l'exécution d'un algorithme
 - Notation :
- Décision = branchement sur plusieurs transitions
 - Notation :



204

Exemple de diagramme d'activité

opération PréparerBoisson de la classe Personne



205

Liens modèles statiques/dynamiques

- Le modèle dynamique définit des séquences de transformation pour les objets
 - Diagramme d'état généralisant pour chaque classe ayant un comportement réactif aux événements les scénarios et collaborations de leurs instances
 - Les variables d'état sont des attributs de l'objet courant
 - Les conditions de déclenchement et les paramètres des actions exploitent les variables d'état et les objets accessibles
 - Diagrammes d'activités associés aux opérations/transitions/méthodes
- Les modèles dynamiques d'une classe sont transmis par héritage aux sous-classes

207

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement



208

Le diagramme d'architecture ou diagramme de structure composite

- Description de la structure interne d'un objet complexe (composants) lors de son exécution (au *run-time* - décrire l'exécution du programme)
- Description de ses points d'interaction avec le reste du système
- Inspiré par l'électronique

209

Port

- Un **port** permet de spécifier les *points d'interactions* d'un objet
 - entre l'objet et son environnement
 - entre l'objet et sa décomposition interne.
- Un port indique les **interfaces** proposées par l'objet (entrées) ainsi que les interfaces requises (sorties)
 - Les ports sont reliés au moyen de **connecteurs** (lignes) sur lesquels transitent des messages
- Un **port comportemental** (*Behaviour port*) est directement associé à la machine à états de la classe qui porte ce port.
 - Tous les messages envoyés sur ce port sont consommés par la machine à états de la classe.



210

Port

- Les ports apparaissent essentiellement dans les diagrammes de **classes** et d'**architecture**.
- Un port peut être placé sur :
 - une classe
 - un part
 - sur le pourtour (frame) d'un diagramme d'architecture

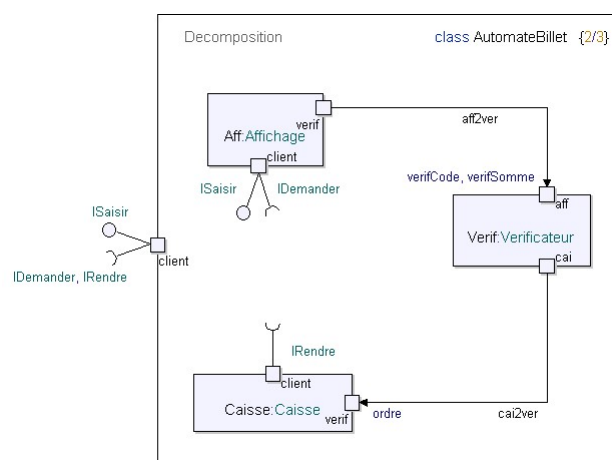
211

Parts

- Désigne une ou plusieurs instances qui compose(nt) une classe
- Un part est représenté dans un diagramme d'**architecture** (nouveau diagramme proposé par UML2.0) aussi appelé **diagramme structure composite**
- Un part représente une ou plusieurs instances d'une classe grâce à des contraintes de multiplicité

212

Exemple de diagramme d'architecture



213

Modélisation UML

Modélisation selon 4 points de vue principaux :

- Aspects statiques du système (*le QUI?*)
 - Description des objets et de leurs relations
 - Modularité, contrats, relations, généricité, héritage
 - Structuration en paquetages
- Vision utilisateur du système (*le QUOI?*)
 - Cas d'utilisation
- Aspects dynamiques du système (*le QUAND?*)
 - Diagramme de séquences (scénarios)
 - Diagramme de collaborations (entre objets)
 - Diagramme d'états-transitions (Harel)
 - Diagramme d'activités
- Vision implantation (*le OU?*)
 - Diagramme de composants et de déploiement

214

Processus de développement avec UML



215